ME 249 Project 2 Report

Comparative Neural Network Modeling: From First-Principles Backpropagation to Keras and Genetic Algorithm Applications in Hybrid Solar–Gas Turbine Systems

Md Elius, Parker Yang, Van P. Carey

Abstract: This project explores the implementation and comparison of neural network modeling approaches for engineering systems. Part 1 develops a first-principles neural network using manually coded backpropagation to model a simple nonlinear system, then compares its performance and training behavior against a Keras implementation using identical network architectures and activation functions. Part 2 extends these concepts to a hybrid solar–fossil fuel gas turbine system, constructing a neural network model to predict the required air-to-fuel ratio and system efficiency under varying inlet temperature, solar heat input, and fuel composition. The study investigates two training methodologies: classical backpropagation and a genetic algorithm (GA) using the pygad package. This evaluates convergence, accuracy, and sensitivity to hyperparameters. The combined analysis highlights the trade-offs between analytical transparency and computational efficiency in modern neural network design for thermodynamic systems.

This work was done as a group project between Md Elius and Parker Yang. The work was divided into several parts. Part 1's first part of compute was done through Parker's compute system and second part of Keras compute was done through Elius's compute system. Part 2 of the project was also similarly divided where first two parts of the project was done on Parker's compute and last part was done on Elius' compute system's conda environment. The write-up of the project report was also divided into each parts and formatting and labeling were done together.

Task 1.1

	First principles	Keras
w01	1.2213539843846632	1.2093952
w02	0.38959118594206515	0.3627017
w03	0.6898258213067975	0.7101982
b1	-0.1601874549651644	-0.1446833
w12	0.7169331511474845	0.7044625
b2	-0.12733496757491838	-0.11254057
w23	0.6967949990401158	0.6816156
b3	0.004865522697557137	0.01967728
	rms = 0.00029173714371156796	mae=loss =0.0125

Table 1.1

Task 1.2

(a) We set epochs = 400 in cell 4 and get error 0.01258 which is less than 0.014. We vary the learning rate from 0.01 to 0.001 and find that for a low learning rate, a large number of epochs are needed to meet the minimum error threshold and vice versa.

(b) The best epoch is number 6 which has the lowest error. The code breaks down after 86 epochs, which means there is no improvement running with high epochs. 6 epochs are sufficient to fulfill the loss criteria which is 0.014.

	First principles	Keras
w01	1.2213539843846632	1.2093952
w02	0.38959118594206515	0.3627017
w03	0.6898258213067975	0.7101982
b1	-0.1601874549651644	-0.1446833

w12	0.7169331511474845	0.7044625
b2	-0.12733496757491838	-0.11254057
w23	0.6967949990401158	0.6816156
b3	0.004865522697557137	0.01967728
	rms = 0.00029173714371156796	mae=loss =0.0125

Table 1.2

(c)

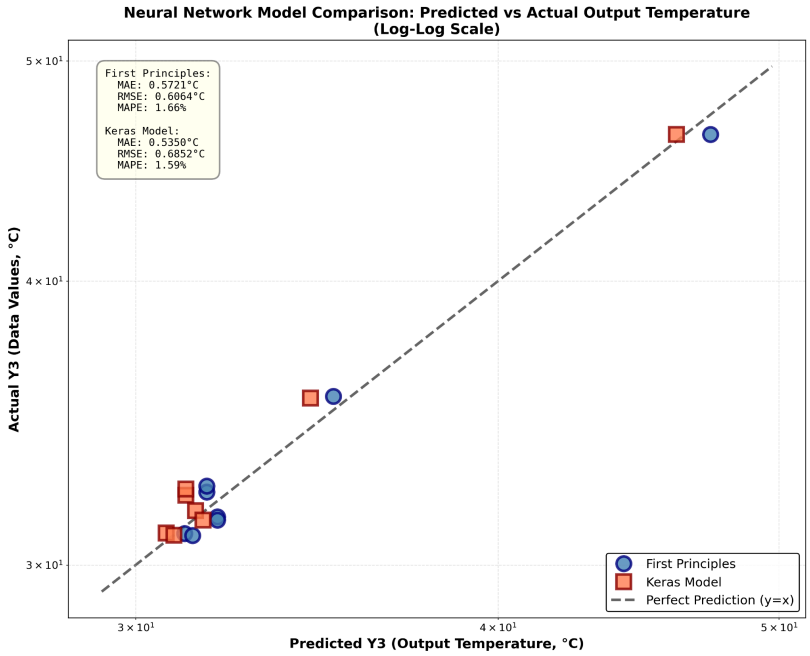


Figure 1.1

The models agree very well overall (both <2% error, very similar R^2), but produce slightly different individual predictions due to different optimization methods. This is completely expected and makes perfect sense, different optimization techniques naturally lead to different solutions, even when starting from identical weights.

(d)

	Original value	20% higher than original
w01	1.23	1.48
w02	0.40	0.48

w03	0.70	0.84
b1	-0.15	-0.18
w12	0.72	0.864
b2	-0.12	-0.144
w23	0.7	0.840
b3	0.01	0.012

Table 1.3

When the starting values are increased by 20%, both the first-principles model and the Keras model require significantly more epochs to reach the same minimum error threshold.
In the first-principles model, achieving an RMS error of 0.00035 with a learning rate of 0.20 originally took only 2 epochs, but with 20% higher starting values, it required 121 epochs to reach the same accuracy.

A similar trend is observed in the Keras model. Under the same setup, the best performance is achieved at epoch 195, compared to just epoch 6 with the original values.

These results indicate that both the first-principles and Keras models are highly sensitive to initial parameter values, and even a modest 20% increase can substantially affect convergence speed.

Part 2

Task 2.1

a)
The new median values for each parameters in *xdata* and *ydata* are the following:

T_1	303.0 K
γ	0.25
$\dot{Q}_s$	1500.0 kW
α	60.36
η_{sys}	0.42

Table 2.1

Task 2.2

b) Our model achieved a value of mean absolute error of `0.04914478957653046`.

Our model continuously improved its mean absolute error every repetition of training.

In addition, Cell 5 in the Code P2.4F25.ipynb provides an example of how to predict values of α & η_{sys}. Figure 2.1 shows the execution of the provided example code set. Title, x-label, and y-label were edited accordingly.

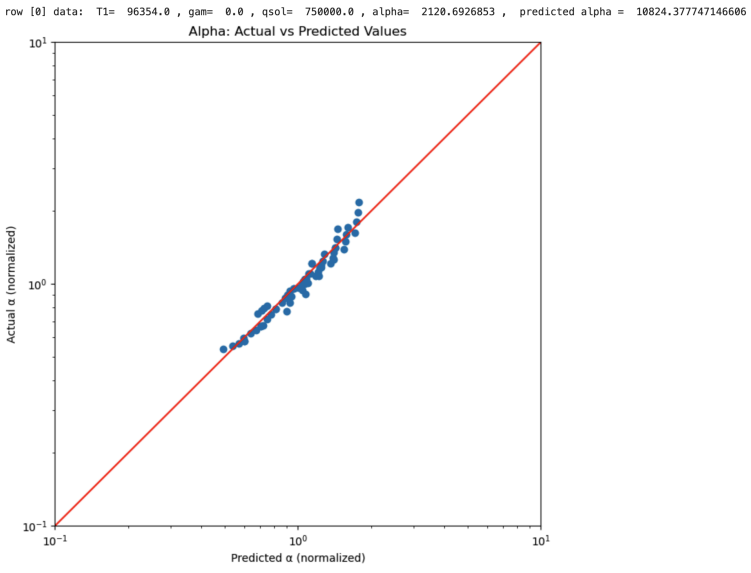


Figure 2.1

Figure 2.2 shows the surface plot for α as a function of T_1 and $\dot{Q}_s$ over the given ranges (268 < T_1 < 318 K & and 500 < $\dot{Q}_s$ < 2500 kW).

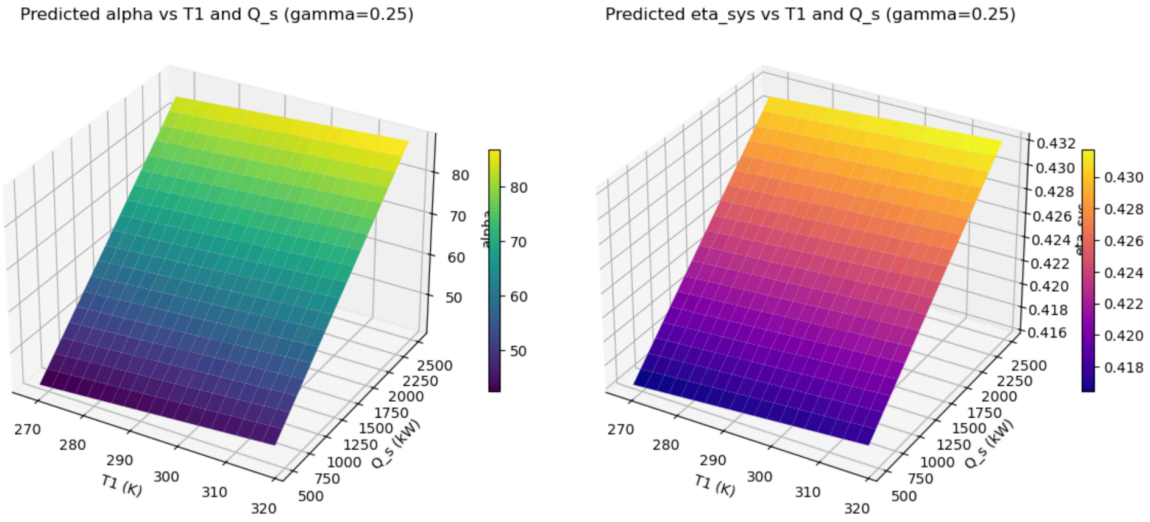


Figure 2.2 surface plot for T_1 and $\dot{Q}_s$

Task 2.3

Task 2.3 was done with a normalized new set of test data (distinct from the training data). Figure ~ and Table ~ were generated using *model.predict* function, comparing the actual and predicted Alpha value.

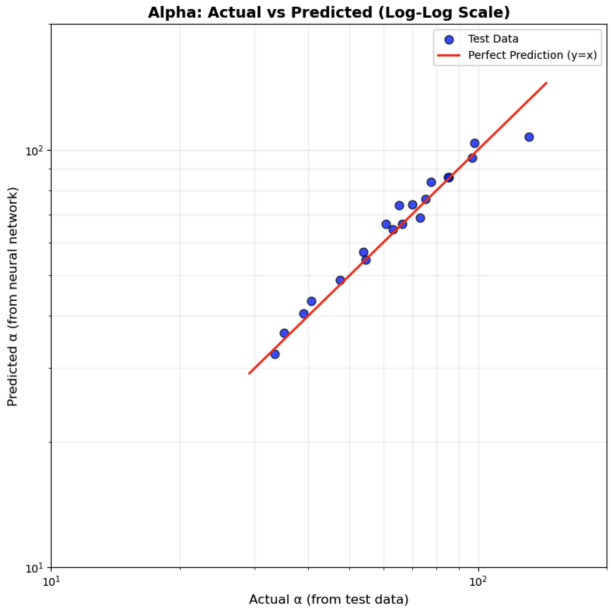


Figure 2.3 Log-log plot of actual vs predicted values of Alpha

RMS deviation for alpha	6.3297

Relative RMS error for alpha	9.36%
Mean Absolute Error for alpha	3.7165

Table 2.2 RMS deviation analysis for Alpha

Task 2.4

(a) The pygad package is installed successfully in our conda environment.

(b) The full dataset from Code 2.4 was transferred to the Code 2.5 Python script, where the Genetic Algorithm with PyGAD was run for 1500 generations. This resulted in a total of 40 × 1500 = 60,000 predictions, making the computation significantly slower. Despite the extensive runtime, the MAE remained relatively high, for instance, in Case 1, the MAE was 0.045 with a corresponding best fitness value of 22.277, indicating that the approach is computationally very expensive and less efficient

Case	1	2	3	4	5	6
NN layers	4,8,4,2	4,8,4,2	4,8,4,2	4,8,4,2	4,5,4,3,2	4,5,4,3,2
Act. Funct.	relu	elu	relu	elu	relu	elu
W,b minval	-0.9	-0.9	-2.9	-0.4	-2.9	-0.9
W,b maxval	0.9	0.9	2.9	0.4	2.9	0.9
Num Solutions	40	40	40	40	40	40
Number of generation (initial)*	1500	1500	1500	1500	1500	1500
Best Fitness value at end:	21.9390	19.24635	22.543746	19.336814	19.3249042	21.492932
Corresponding best combined mae:	0.045092713	0.051957868	0.04435819	0.05171	0.05174669	0.046526913

Table 2.3

(c)

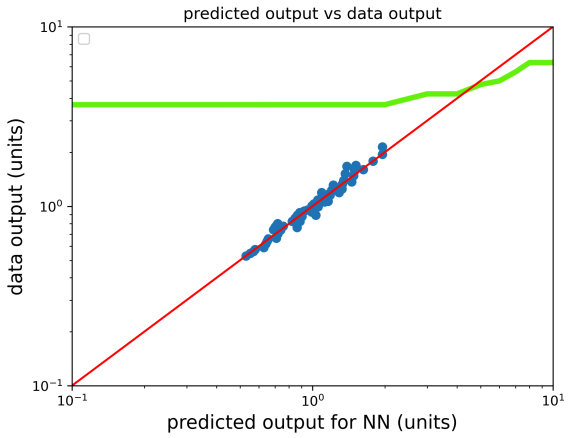


Figure 2.4 Predicted output vs data output for case 1 (1500 generations)

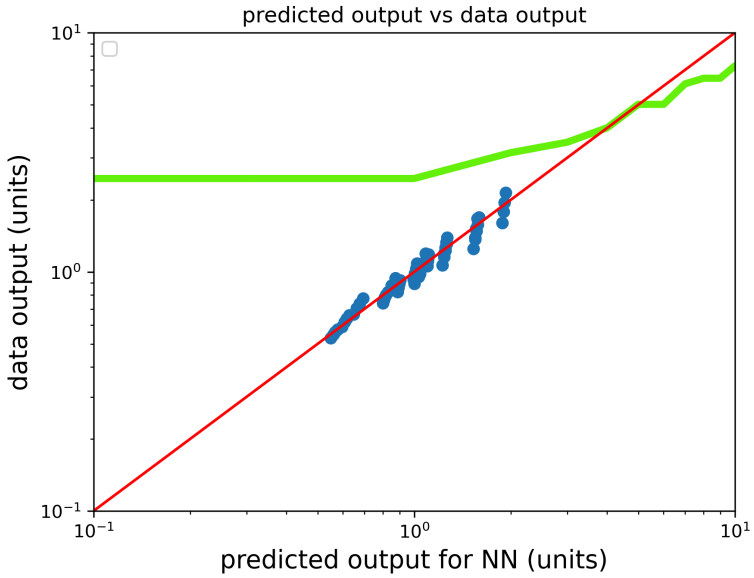


Figure 2.5: Predicted output vs data output for case 2 (1500 generations)

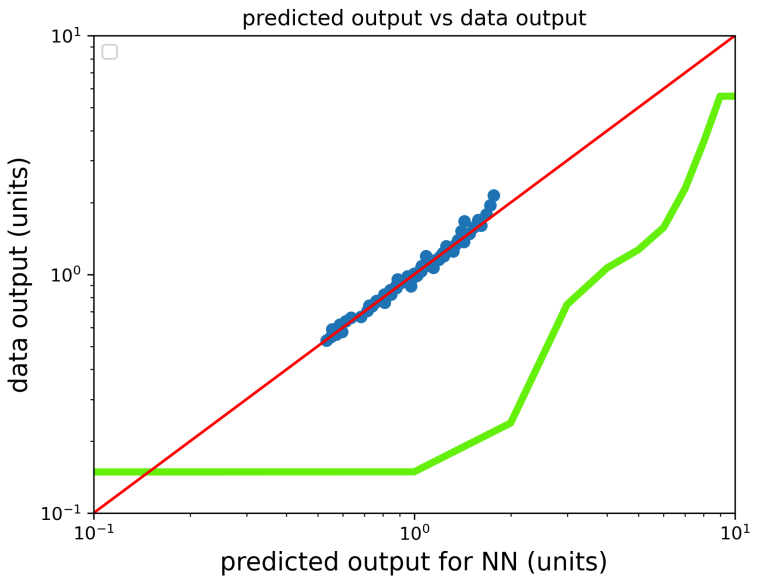


Figure 2.6: Predicted output vs data output for case 3 (for 1500 generations)

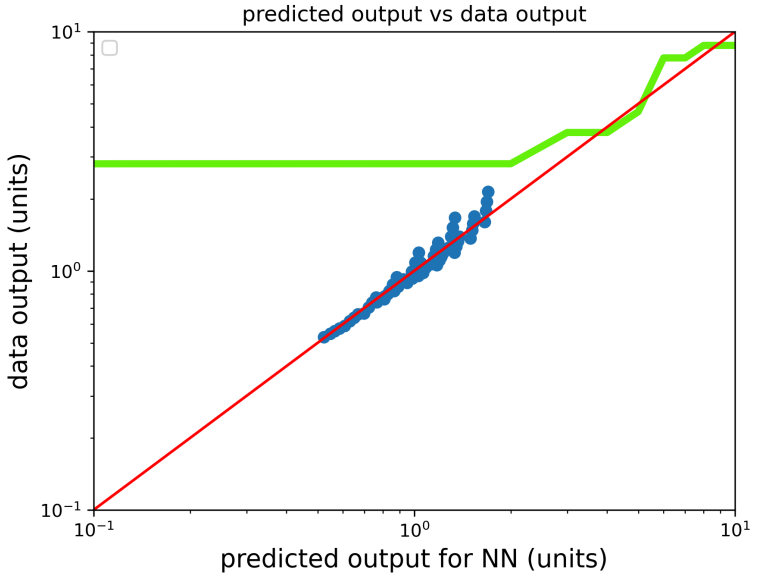


Figure 2.7: Predicted output vs data output for case 4 (for 1500 generations)

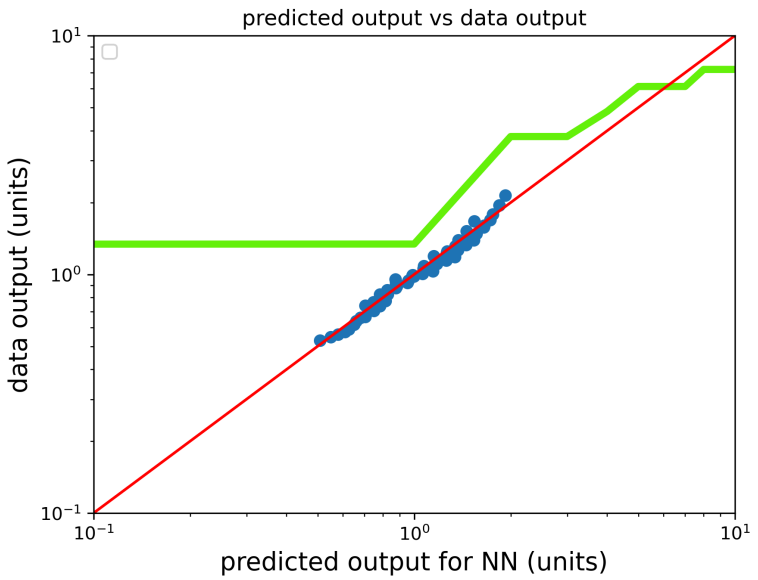


Figure 2.8: Predicted output vs data output for case 5 (for 1500 generations)

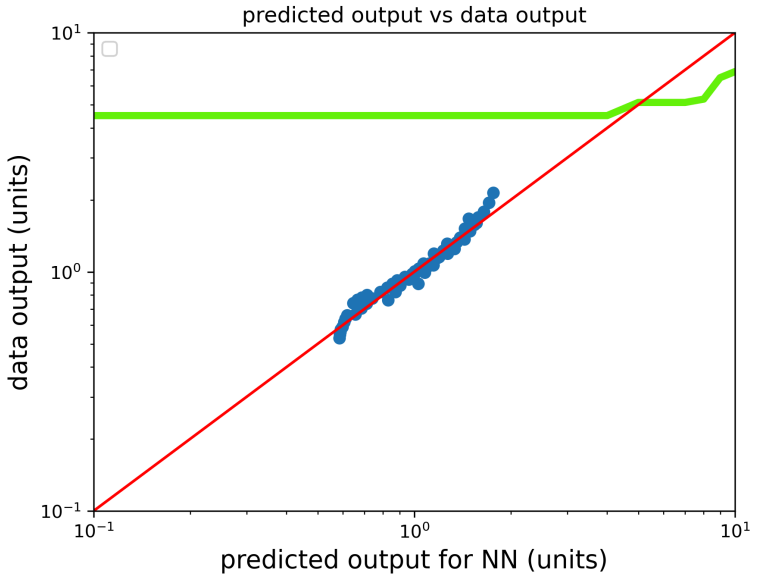


Figure 2.9: Predicted output vs data output for case 6 (for 1500 generations)

Task 2.4 was completed using a batch job script. We ran 6 jobs for 6 cases. Job script is given code in the appendix section.

All six cases used 1500 generations with 40 solutions (60,000 predictions). Cases 1, 3, and 6 achieved the lowest MAE values (~0.045, 0.044, and 0.047), with Case 3 performing best (MAE = 0.0444, fitness = 22.54). Cases 2, 4, and 5 had higher errors (~0.052). While most runs took about 130 minutes, Case 4 required 160 minutes due to slower ELU convergence. Overall, Case 3 (ReLU, W/b = −2.9 to 2.9) gave the best accuracy and efficiency.

The backpropagation model in task 2.2 achieved a smaller mean absolute error (0.0248 vs 0.0451) in less than a minute and reached the threshold error of 0.03 after about 395 epochs, while the Genetic Algorithm with PyGAD required roughly 130 minutes to run 1500 generations with 40 solutions each (a total of 60,000 predictions) to reach an error of 0.0451. Backpropagation converged smoothly and early-stopped at epoch 359, whereas the GA evolved much more slowly since it depends on population fitness evaluations rather than gradient information. Overall, backpropagation is significantly faster, computationally efficient, and more effective for smooth, differentiable problems, making it the clearly preferred training method in this case.

Conclusions: This project demonstrated and compared two neural network training approaches, backpropagation and a genetic algorithm, applied to a hybrid solar, gas turbine system model. The backpropagation method achieved faster convergence, lower mean absolute error (0.0248), and efficient early stopping, completing training in under a minute. In contrast, the Genetic Algorithm with PyGAD required around 130–160 minutes and 60,000 predictions to reach a higher error (~0.045). Overall, backpropagation proved to be far more effective and computationally efficient for smooth, differentiable systems, while the GA offered insight into heuristic optimization but at a significant computational cost

Appendix

Job script bashrc for task 2.4 (6 jobs run for 6 cases)

```
#!/bin/bash
#SBATCH --job-name=Compilereal
#SBATCH --partition=savio3
#SBATCH --qos=aiolos_savio3_normal
#SBATCH --account=co_aiolos
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=40
#SBATCH --time=03:59:59
#SBATCH --mail-type=all
#SBATCH --mail-user=m_elius@berkeley.edu
#SBATCH --output=job_%j.out
#SBATCH --error=job_%j.err

# 1) Start clean and go to your project dir
module purge
cd /global/scratch/users/melius/Fall_2025/ME249_Machine_Learning/me249/prj2

# 2) Activate your conda env
```

```
source ~/.bashrc
conda activate my_kernel

# 3) Ensure Conda's libstdc++ is found before system one (fixes GLIBCXX errors)
export LD_LIBRARY_PATH="${CONDA_PREFIX}/lib:${LD_LIBRARY_PATH}"

# 4) Avoid accidental Python contamination + oversubscription
unset PYTHONHOME PYTHONPATH
export PYTHONNOUSERSITE=1
export OMP_NUM_THREADS=1
export OPENBLAS_NUM_THREADS=1
export MKL_NUM_THREADS=1
export NUMEXPR_NUM_THREADS=1

# 5) (Optional) Convert notebook to script
jupyter nbconvert --to script CodeP2.5F25.ipynb

# 6) (Optional) quick diagnostics to the job log
python - <<'PY'
import sys, subprocess
print("Python:", sys.version)
try:
    import pandas._libs.window.aggregations as m
    print("Pandas ext:", m.__file__)
    subprocess.run(["bash","-lc", f"ldd {m.__file__} | grep libstdc++ || true"])
except Exception as e:
    print("Pandas import test failed:", e)
PY

# 7) Run your code
python CodeP2.5F25.py
```

In [1]:
```
'''#Intro to Neural Network Modeling
# Python Neural Network Model of Spray Cooling Test System

>>>>> start CodeP2.1F25
    V.P. Carey, ME249, Fall 2025'''

# version 3 print function
from __future__ import print_function

# import math, numpy and other usefuk packages
import math
import numpy

%matplotlib inline
# importing the required module
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [10, 8] # for square canvas


#assembling data array
#store array where rows are data vectors [x01, x02, x03, y3]
xydata = []

xydata = [[20./20.2, 13.0/14.5, 310.8/308.0, 30.99/32.4], [20./20.2, 14.5/14
xydata.append([20./20.2, 15.3/14.5, 306.0/308.0, 31.7/32.4])
xydata.append([20.2/20.2, 13.0/14.5, 310.8/308.0, 30.92/32.4])
xydata.append([20./20.2, 14.5/14.5, 308.0/308.0, 32.4/32.4])
xydata.append([20./20.2, 15.3/14.5, 306.0/308.0, 31.4/32.4])
xydata.append([24./20.2, 13.0/14.5, 310.8/308.0, 35.53/32.4])
xydata.append([36./20.2, 14.4/14.5, 308.1/308.0, 46.41/32.4])
print (xydata)

#set original starting values
w01n =  1.23
w02n =  0.40
w03n =  0.70
b1n =  -0.15
w12n =  0.72
b2n =  -0.12
w23n =  0.7
b3n =  0.01

# #set starting values with 20 higher than origianl value
# w01n =  1.23*1.2
# w02n =  0.40*1.2
# w03n =  0.70*1.2
# b1n =  -0.15*1.2
# w12n =  0.72*1.2
# b2n =  -0.12*1.2
# w23n =  0.7*1.2
# b3n =  0.01*1.2

#start of batch loop
epoch_count = 0
```

```
for k in range (0,200):
    icount = 0
    #initialize error and derivative parameters
    E3ti = 0.
    dE3da3 = 0.
    dE3dw01ti = 0.
    dE3dw02ti = 0.
    dE3dw03ti = 0.
    dE3db1ti = 0.
    dE3dw12ti = 0.
    dE3db2ti = 0.
    dE3dw23ti = 0.
    dE3db3ti = 0.

    w01 = w01n
    w02 = w02n
    w03 = w03n
    b1 = b1n
    w12 = w12n
    b2 = b2n
    w23 = w23n
    b3 = b3n

    #doing calcuations for each data point
    for i in range(0,8):
        #compute activation functions and their derivatives
        z1 = w01*xydata[i][0]+w02*xydata[i][1]+w03*xydata[i][2]+b1
        sig1 = z1
        sigp1 = 1.0
        if z1 < 0.0:
            sig1 = math.exp(z1) - 1.0
            sigp1 = math.exp(z1)
        a1 = sig1

        z2 = w12*a1+b2
        sig2 = z2
        sigp2 = 1.0
        if z2 < 0.0:
            sig2 = math.exp(z2) - 1.0
            sigp2 = math.exp(z2)
        a2 = sig2

        z3 = w23*a2+b3
        sig3 = z3
        sigp3 = 1.0
        if z3 < 0.0:
            sig3 = math.exp(z3) - 1.0
            sigp3 = math.exp(z3)
        a3 = sig3


        #compute derivatives for backpropagation
        #add to sum for batch average calculation
        E3ti = E3ti +(a3 - xydata[i][3])*(a3 - xydata[i][3])
        dE3da3 = 2.*(a3 - xydata[i][3])
```

```
        dE3dw01ti = dE3dw01ti + dE3da3*sigp3*w23*sigp2*w12*sigp1*xydata[i][0
        dE3dw02ti = dE3dw02ti + dE3da3*sigp3*w23*sigp2*w12*sigp1*xydata[i][1
        dE3dw03ti = dE3dw03ti + dE3da3*sigp3*w23*sigp2*w12*sigp1*xydata[i][2
        dE3db1ti =  dE3db1ti +  dE3da3*sigp3*w23*sigp2*w12*sigp1

        dE3dw12ti = dE3dw12ti + dE3da3*sigp3*w23*sigp2*a1
        dE3db2ti = dE3db1ti +  dE3da3*sigp3*sigp2

        dE3dw23ti = dE3dw23ti + dE3da3*sigp3*a2
        dE3db3ti = dE3db3ti + dE3da3*sigp3

        icount = i + 1
        # end  calculations for each data point in batch

    #compute batch averaged values
    E3 = E3ti/icount
    dE3dw01 = dE3dw01ti/icount
    dE3dw02 = dE3dw02ti/icount
    dE3dw03 = dE3dw03ti/icount
    dE3db1 = dE3db1ti/icount
    dE3dw12 = dE3dw12ti/icount
    dE3db2 = dE3db2ti/icount
    dE3dw23 = dE3dw23ti/icount
    dE3db3 = dE3db3ti/icount

    #set gam = learning rate
    gam = 0.03
    # gam = 0.03
    if E3 < 0.07:
        gam = 0.2

    w01n = w01 + gam*(-E3)/dE3dw01
    w02n = w02 + gam*(-E3)/dE3dw02
    w03n = w03 + gam*(-E3)/dE3dw03
    b1n = b1 + gam*(-E3)/dE3db1
    w12n = w12 + gam*(-E3)/dE3dw12
    b2n = b2 + gam*(-E3)/dE3db2

    w23n = w23 + gam*(-E3)/dE3dw23
    b3n = b3 + gam*(-E3)/dE3db3
    epoch_count = k+1
    #printing for each iteration
    print ('last w01, w02, w03, w12, w23:')
    print ('last b1, b2, b3:')
    print (w01, w02, w03, w12, w23)
    print (b1, b2, b3)
    print ('E3 = ', E3, 'icount =', icount)
    print ('next ws:', w01n, w02n, w03n, w12n, w23n)
    print ('next bs:', b1n, b2n, b3n)

    #quit if squared error is below target
    if E3 < 0.00035:
        break


print ('last w01, w02, w03, w12, w23:')
```

```
print ('last b1, b2, b3:')
print (w01, w02, w03, w12, w23)
print (b1, b2, b3)
print('epoch number', epoch_count)
#decomment print statements below if you want to print neuron outputs
#print ('z1 =', z1)
#print ('a1 =', a1)
#print ('z2 =', z2)
#print ('a2 =', a2)
#print ('z3 =', z3)
#print ('a3 =', a3)

#print comparison of data and trained network predictions
# restore raw data values
xydatar = [[20., 13.0, 310.8, 30.97], [20., 14.5, 308.0, 32.3]]
xydatar.append([20., 15.3, 306.0, 31.5])
xydatar.append([20.2, 13.0, 310.8, 30.91])
xydatar.append([20., 14.5, 308.0, 32.5])
xydatar.append([20., 15.3, 306.0, 31.4])
xydatar.append([24., 13.0, 310.8, 35.59])
xydatar.append([36., 14.4, 308.0, 46.41])

print ('Tdbin, Twbin, qdot, Tdbout, ypredicted:')
for i in range(0,8):
    z1 = w01*xydata[i][0]+w02*xydata[i][1]+w03*xydata[i][2]+b1
    sig1 = z1
    sigp1 = 1.0
    if z1 < 0.0:
        sig1 = math.exp(z1) - 1.0
        sigp1 = math.exp(z1)
    a1 = sig1

    z2 = w12*a1+b2
    sig2 = z2
    sigp2 = 1.0
    if z2 < 0.0:
        sig2 = math.exp(z2) - 1.0
        sigp2 = math.exp(z2)
    a2 = sig2

    z3 = w23*a2+b3
    sig3 = z3
    sigp3 = 1.0
    if z3 < 0.0:
        sig3 = math.exp(z3) - 1.0
        sigp3 = math.exp(z3)
    a3 = sig3

    print (xydatar[i][0], xydatar[i][1], xydatar[i][2], xydatar[i][3], a3*32
```

```
[[0.9900990099009901, 0.896551724137931, 1.009090909090909, 0.95648148148148
15], [0.9900990099009901, 1.0, 1.0, 0.9938271604938272], [0.990099009900990
1, 1.0551724137931036, 0.9935064935064936, 0.9783950617283951], [1.0, 0.8965
51724137931, 1.009090909090909, 0.954320987654321], [0.9900990099009901, 1.
0, 1.0, 1.0], [0.9900990099009901, 1.0551724137931036, 0.9935064935064936,
0.9691358024691358], [1.188118811881188, 0.896551724137931, 1.00909090909090
9, 1.096604938271605], [1.7821782178217822, 0.993103448275862, 1.00032467532
46755, 1.4324074074074074]]
last w01, w02, w03, w12, w23:
last b1, b2, b3:
1.23 0.4 0.7 0.72 0.7
-0.15 -0.12 0.01
E3 =  0.002251781415924418 icount = 8
next ws: 1.2213539843846632 0.38959118594206515 0.6898258213067975 0.7169331
511474845 0.6967949990401158
next bs: -0.1601874549651644 -0.12712722965855752 0.004865522697557137
last w01, w02, w03, w12, w23:
last b1, b2, b3:
1.2213539843846632 0.38959118594206515 0.6898258213067975 0.7169331511474845
0.6967949990401158
-0.1601874549651644 -0.12712722965855752 0.004865522697557137
E3 =  0.0002928160870044705 icount = 8
next ws: 1.212116804781277 0.3744781911065515 0.6743412417615255 0.713202357
8258627 0.6929383349455349
next bs: -0.17561548183776973 -0.13240278759093024 -0.0028416319681919587
last w01, w02, w03, w12, w23:
last b1, b2, b3:
1.2213539843846632 0.38959118594206515 0.6898258213067975 0.7169331511474845
0.6967949990401158
-0.1601874549651644 -0.12712722965855752 0.004865522697557137
epoch number 2
Tdbin, Twbin, qdot, Tdbout, ypredicted:
20.0 13.0 310.8 30.97 31.187674466338166
20.0 14.5 308.0 32.3 31.738492841534573
20.0 15.3 306.0 31.5 32.01389557670518
20.2 13.0 310.8 30.91 31.383400627268312
20.0 14.5 308.0 32.5 31.738492841534573
20.0 15.3 306.0 31.4 32.01389557670518
24.0 13.0 310.8 35.59 35.1021976849411
36.0 14.4 308.0 46.41 47.35672275614189
```

Keras Version

In [ ]:

In [1]:
```
%reset -f
'''>>>>> start CodeP2.2F25
    V.P. Carey ME249, Fall 2025

Intro to Neural Network Modeling
Keras model for comparison with first principles model'''

#import useful packages
import keras
import pandas as pd
from keras.models import Sequential
import numpy as np
import keras.backend as kb
import tensorflow as tf
#the following 2 lines are only needed for Mac OS machines
import os
os.environ['KMP_DUPLICATE_LIB_OK']='True'

#raw data in dictionary form x01, x02, x03, y3
my_dict = {
    'x01' : [20., 20., 20., 20.2, 20., 20.2, 24.0, 36.],
    'x02' : [13., 14.5, 15.3, 13., 14.5, 15.3, 13., 14.4],
    'x03' : [310.8, 308.0, 306.0, 310.8, 308.0, 306.0, 310.8, 308.1],
    'y3' : [30.99, 32.2, 31.7, 30.92, 32.4, 31.4, 35.53, 46.41]
}
#normalized inputs in array
xdata = []
xdata = [[20./20.2, 13.0/14.5, 310.8/308.0], [20./20.2, 14.5/14.5, 308.0/308
xdata.append([20./20.2, 15.3/14.5, 306.0/308.0])
xdata.append([20.2/20.2, 13.0/14.5, 310.8/308.0])
xdata.append([20./20.2, 14.5/14.5, 308.0/308.0])
xdata.append([20.2/20.2, 15.3/14.5, 306.0/308.0])
xdata.append([24./20.2, 13.0/14.5, 310.8/308.0])
xdata.append([36./20.2, 14.4/14.5, 308.1/308.0])

#data frame
df = pd.DataFrame(my_dict)
#devide by the median to normalize
df.x01= df.x01/20.2
df.x02= df.x02/14.5
df.x03= df.x03/308.0
#normalize output array
df.y3= df.y3/32.401
df.head
print (df.x01, df.x02, df.x03, df.y3)

xarray= np.array(xdata)
print (xdata)
print (xarray)
```

```
0    0.990099
1    0.990099
2    0.990099
3    1.000000
4    0.990099
5    1.000000
6    1.188119
7    1.782178
Name: x01, dtype: float64 0    0.896552
1    1.000000
2    1.055172
3    0.896552
4    1.000000
5    1.055172
6    0.896552
7    0.993103
Name: x02, dtype: float64 0    1.009091
1    1.000000
2    0.993506
3    1.009091
4    1.000000
5    0.993506
6    1.009091
7    1.000325
Name: x03, dtype: float64 0    0.956452
1    0.993796
2    0.978365
3    0.954292
4    0.999969
5    0.969106
6    1.096571
7    1.432363
Name: y3, dtype: float64
[[0.9900990099009901, 0.896551724137931, 1.009090909090909], [0.990099009900
9901, 1.0, 1.0], [0.9900990099009901, 1.0551724137931036, 0.993506493506493
6], [1.0, 0.896551724137931, 1.009090909090909], [0.9900990099009901, 1.0,
1.0], [1.0, 1.0551724137931036, 0.9935064935064936], [1.188118811881188, 0.8
96551724137931, 1.009090909090909], [1.7821782178217822, 0.993103448275862,
1.0003246753246755]]
[[0.99009901 0.89655172 1.00909091]
 [0.99009901 1.         1.        ]
 [0.99009901 1.05517241 0.99350649]
 [1.         0.89655172 1.00909091]
 [0.99009901 1.         1.        ]
 [1.         1.05517241 0.99350649]
 [1.18811881 0.89655172 1.00909091]
 [1.78217822 0.99310345 1.00032468]]
```

In [ ]:

In [2]:
```
# define model

#As seen below, we have created three dense layers each with just one neuron
#A dense layer is a layer in neural network that's fully connected.
#In other words, all the neurons in one layer are connected to all other neu
#In the first layer, we need to provide the input shape, which is 3 in this
```

```
#The activation function we have chosen is ReLU, which stands for rectified

from keras import backend as K
#initialize weights with values between -0.2 and 1.2
initializer = keras.initializers.RandomUniform(minval= -0.2, maxval=1.2)

# define three layer model with one neuron in each layer
model = keras.Sequential([
    # keras.layers.Dense(1, activation=K.elu, input_shape=[3],  kernel_initi
    # keras.layers.Dense(1, activation=K.elu,  kernel_initializer=initialize
    # keras.layers.Dense(1, activation=K.elu,  kernel_initializer=initialize
    keras.layers.Dense(1, activation='elu', input_shape=[3],  kernel_initial
    keras.layers.Dense(1, activation='elu',  kernel_initializer=initializer,
    keras.layers.Dense(1, activation='elu',  kernel_initializer=initializer,
  ])
model.summary()

# #set original starting values to those used in first principles model
# w01n =  1.23
# w02n =  0.40
# w03n =  0.70
# b1n =  -0.15
# w12n =  0.72
# b2n =  -0.12
# w23n =  0.7
# b3n =  0.01

#set starting values to those used in first principles model with 20% higher
w01n =  1.23*1.2
w02n =  0.40*1.2
w03n =  0.70*1.2
b1n =  -0.15*1.2
w12n =  0.72*1.2
b2n =  -0.12*1.2
w23n =  0.7*1.2
b3n =  0.01*1.2


weights0 =  [[ w01n], [w02n], [ w03n]]
w0array= np.array(weights0)
print(np.shape(w0array))
bias0 = [b1n]
bias0array= np.array(bias0)
L0=[]
L0.append(w0array)
L0.append(bias0array)
model.layers[0].set_weights(L0)

weights1 =  [[ w12n]]
w1array= np.array(weights1)
print(np.shape(w1array))
bias1 = [b2n]
bias1array= np.array(bias1)
L1=[]
L1.append(w1array)
L1.append(bias1array)
```

```
model.layers[1].set_weights(L1)

weights2 =  [[ w23n]]
w2array= np.array(weights2)
print(np.shape(w2array))
bias2 = [b3n]
bias2array= np.array(bias2)
L2=[]
L2.append(w2array)
L2.append(bias2array)
model.layers[2].set_weights(L2)
```

```
/global/home/users/melius/.conda/envs/my_kernel/lib/python3.11/site-package
s/keras/src/layers/core/dense.py:92: UserWarning: Do not pass an `input_shap
e`/`input_dim` argument to a layer. When using Sequential models, prefer usi
ng an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2025-10-28 16:06:52.274945: E external/local_xla/xla/stream_executor/cuda/cu
da_driver.cc:266] failed call to cuInit: CUDA_ERROR_NO_DEVICE: no CUDA-capab
le device is detected
```

Model: "sequential"

Layer (type)	Output Shape	Par
dense_one (Dense)	(None, 1)	
dense_two (Dense)	(None, 1)	
dense_three (Dense)	(None, 1)	

Total params: 8 (32.00 B)

Trainable params: 8 (32.00 B)

Non-trainable params: 0 (0.00 B)

```
(3, 1)
(1, 1)
(1, 1)
```

In [24]:
```
#We're using RMSprop as our optimizer here. RMSprop stands for Root Mean Squ
#It's one of the most popular gradient descent optimization algorithms for d
#RMSprop is an optimizer that's reliable and fast.
#We're compiling the mode using the model.compile function. The loss functio
#is mean absolute error. After the compilation of the model, we'll use the f

#Running model.fit successive times extends the calculation to addtional epo

# rms = keras.optimizers.RMSprop(0.0035)
rms = keras.optimizers.RMSprop(0.030)
model.compile(loss='mean_absolute_error',optimizer=rms)
```

In [25]:
```
#After the compilation of the model, we'll use the fit method with 500 epoch
#I started with epochs value of 100 and then tested the model after training
#The prediction was not that good. Then I modified the number of epochs to 2
#Accuracy had improved slightly, but figured I'd give it one more try. Final
#I found acceptable prediction accuracy.
```

```
#The fit method takes three parameters; namely, x, y, and number of epochs.
#During model training, if all the batches of data are seen by the model onc
#we say that one epoch has been completed.

# Add an early stopping callback
es = tf.keras.callbacks.EarlyStopping(
    monitor='loss',
    mode='min',
    patience = 80,
    restore_best_weights = True,
    verbose=1)
# Add a checkpoint where loss is minimum, and save that model
# mc = tf.keras.callbacks.ModelCheckpoint('best_model.SB', monitor='loss',
mc = tf.keras.callbacks.ModelCheckpoint('best_model.keras', monitor='loss',
                     mode='min',  verbose=1, save_best_only=True)

historyData = model.fit(xarray,df.y3,epochs=400,callbacks=[es])

loss_hist = historyData.history['loss']
#The above line will return a dictionary, access it's info like this:
best_epoch = np.argmin(historyData.history['loss']) + 1
print ('best epoch = ', best_epoch)
print('smallest loss =', np.min(loss_hist))
```

```
Epoch 1/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 397ms/step - loss: 0.8587
Epoch 2/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0604
Epoch 3/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.3628
Epoch 4/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0596
Epoch 5/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.2308
Epoch 6/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1426
Epoch 7/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1629
Epoch 8/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1538
Epoch 9/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.1171
Epoch 10/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1673
Epoch 11/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0819
Epoch 12/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1814
Epoch 13/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0527
Epoch 14/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1957
Epoch 15/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0336
Epoch 16/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1051
Epoch 17/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1382
Epoch 18/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1199
Epoch 19/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1113
Epoch 20/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1346
Epoch 21/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0876
Epoch 22/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1492
Epoch 23/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0659
Epoch 24/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1637
Epoch 25/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0458
Epoch 26/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1780
Epoch 27/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0324
Epoch 28/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0944
```

```
Epoch 29/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1306
Epoch 30/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1093
Epoch 31/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1084
Epoch 32/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1239
Epoch 33/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0879
Epoch 34/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1384
Epoch 35/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0686
Epoch 36/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1527
Epoch 37/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0502
Epoch 38/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1669
Epoch 39/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0346
Epoch 40/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0854
Epoch 41/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1342
Epoch 42/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1002
Epoch 43/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1131
Epoch 44/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1148
Epoch 45/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0934
Epoch 46/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1291
Epoch 47/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0747
Epoch 48/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1434
Epoch 49/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0569
Epoch 50/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1575
Epoch 51/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0396
Epoch 52/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1714
Epoch 53/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0287
Epoch 54/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0920
Epoch 55/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1222
Epoch 56/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1066
```

```
Epoch 57/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1019
Epoch 58/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1210
Epoch 59/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0828
Epoch 60/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1353
Epoch 61/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0647
Epoch 62/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1494
Epoch 63/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0472
Epoch 64/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1633
Epoch 65/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0318
Epoch 66/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1325
Epoch 67/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0697
Epoch 68/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1467
Epoch 69/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0515
Epoch 70/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1607
Epoch 71/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0345
Epoch 72/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1298
Epoch 73/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0733
Epoch 74/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1440
Epoch 75/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0546
Epoch 76/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1580
Epoch 77/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0368
Epoch 78/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1719
Epoch 79/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0255
Epoch 80/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0938
Epoch 81/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1175
Epoch 82/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1082
Epoch 83/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0970
Epoch 84/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1225
```

```
Epoch 85/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0778
Epoch 86/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1366
Epoch 87/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0596
Epoch 88/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1505
Epoch 89/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0421
Epoch 90/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1643
Epoch 91/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0277
Epoch 92/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0870
Epoch 93/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1222
Epoch 94/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1014
Epoch 95/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.1019
Epoch 96/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1156
Epoch 97/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0829
Epoch 98/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1296
Epoch 99/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0648
Epoch 100/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1434
Epoch 101/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0476
Epoch 102/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1571
Epoch 103/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0311
Epoch 104/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1274
Epoch 105/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0690
Epoch 106/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1413
Epoch 107/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0510
Epoch 108/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1551
Epoch 109/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0338
Epoch 110/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1687
Epoch 111/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0222
Epoch 112/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0930
```

```
Epoch 113/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1121
Epoch 114/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1072
Epoch 115/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0921
Epoch 116/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1212
Epoch 117/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0733
Epoch 118/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1351
Epoch 119/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0555
Epoch 120/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1488
Epoch 121/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0384
Epoch 122/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1623
Epoch 123/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0240
Epoch 124/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0873
Epoch 125/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1160
Epoch 126/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1015
Epoch 127/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0961
Epoch 128/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.1154
Epoch 129/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0775
Epoch 130/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1292
Epoch 131/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0597
Epoch 132/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1428
Epoch 133/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0428
Epoch 134/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1563
Epoch 135/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0268
Epoch 136/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1275
Epoch 137/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0633
Epoch 138/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1412
Epoch 139/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0457
Epoch 140/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1547
```

```
Epoch 141/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0287
Epoch 142/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1681
Epoch 143/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0187
Epoch 144/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0437
Epoch 145/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.1667
Epoch 146/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0581
Epoch 147/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1448
Epoch 148/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0723
Epoch 149/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1247
Epoch 150/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0863
Epoch 151/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1059
Epoch 152/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1000
Epoch 153/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0882
Epoch 154/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1136
Epoch 155/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0711
Epoch 156/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1270
Epoch 157/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0547
Epoch 158/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1403
Epoch 159/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0388
Epoch 160/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1535
Epoch 161/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0237
Epoch 162/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1257
Epoch 163/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0593
Epoch 164/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1391
Epoch 165/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0423
Epoch 166/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1524
Epoch 167/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0259
Epoch 168/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1655
```

```
Epoch 169/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0170
Epoch 170/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0446
Epoch 171/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.1604
Epoch 172/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0588
Epoch 173/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1391
Epoch 174/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0727
Epoch 175/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1194
Epoch 176/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0865
Epoch 177/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1010
Epoch 178/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1000
Epoch 179/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0836
Epoch 180/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1134
Epoch 181/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0668
Epoch 182/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1266
Epoch 183/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0507
Epoch 184/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1397
Epoch 185/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0350
Epoch 186/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1527
Epoch 187/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0203
Epoch 188/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1257
Epoch 189/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0549
Epoch 190/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1389
Epoch 191/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0381
Epoch 192/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1520
Epoch 193/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0220
Epoch 194/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1649
Epoch 195/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0150
Epoch 196/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0469
```

```
Epoch 197/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1533
Epoch 198/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0610
Epoch 199/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1322
Epoch 200/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0747
Epoch 201/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.1128
Epoch 202/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0883
Epoch 203/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0947
Epoch 204/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1016
Epoch 205/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0775
Epoch 206/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1148
Epoch 207/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0610
Epoch 208/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1278
Epoch 209/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0451
Epoch 210/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1407
Epoch 211/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0297
Epoch 212/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1535
Epoch 213/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0168
Epoch 214/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0826
Epoch 215/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1042
Epoch 216/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0962
Epoch 217/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0855
Epoch 218/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1095
Epoch 219/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0680
Epoch 220/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1226
Epoch 221/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0513
Epoch 222/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1355
Epoch 223/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0353
Epoch 224/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1483
```

```
Epoch 225/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0198
Epoch 226/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1609
Epoch 227/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0174
Epoch 228/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0170
Epoch 229/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0278
Epoch 230/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1809
Epoch 231/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0418
Epoch 232/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1567
Epoch 233/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0556
Epoch 234/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1351
Epoch 235/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0692
Epoch 236/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1154
Epoch 237/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0826
Epoch 238/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0972
Epoch 239/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0957
Epoch 240/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0799
Epoch 241/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1087
Epoch 242/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0635
Epoch 243/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.1215
Epoch 244/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0476
Epoch 245/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1342
Epoch 246/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0323
Epoch 247/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1467
Epoch 248/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0200
Epoch 249/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0653
Epoch 250/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1206
Epoch 251/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0788
Epoch 252/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.1015
```

```
Epoch 253/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0920
Epoch 254/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0836
Epoch 255/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1051
Epoch 256/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0667
Epoch 257/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1179
Epoch 258/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0506
Epoch 259/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1306
Epoch 260/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0350
Epoch 261/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1431
Epoch 262/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0224
Epoch 263/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0623
Epoch 264/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1226
Epoch 265/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0757
Epoch 266/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.1034
Epoch 267/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0889
Epoch 268/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0854
Epoch 269/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1019
Epoch 270/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0685
Epoch 271/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1147
Epoch 272/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0524
Epoch 273/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.1273
Epoch 274/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0368
Epoch 275/400
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.1398
Epoch 275: early stopping
Restoring model weights from the end of the best epoch: 195.
best epoch =  195
smallest loss = 0.014987409114837646
```

In [26]:

```
from __future__ import print_function
#For results of training network:

#keras.layer.get_weights() function retrieves weight values
first_layer_weights = model.layers[0].get_weights()[0]
```

```
w01 = first_layer_weights[0][0]
w02 = first_layer_weights[1][0]
w03 = first_layer_weights[2][0]
first_layer_bias  = model.layers[0].get_weights()[1]
b1 = first_layer_bias
second_layer_weights = model.layers[1].get_weights()[0]
w12 = second_layer_weights[0][0]
second_layer_bias  = model.layers[1].get_weights()[1]
b2 = second_layer_bias
third_layer_weights = model.layers[2].get_weights()[0]
w23 = third_layer_weights[0][0]
third_layer_bias  = model.layers[2].get_weights()[1]
b3 = third_layer_bias

#print weights and biases
print (first_layer_weights)
print ('w01 = ', w01, 'w02 = ', w02, 'w03 = ', w03)
print (first_layer_bias)
print ('b1 = ', b1)
print (second_layer_weights)
print ('w12 = ', w12)
print (second_layer_bias)
print ('b2 = ', b2)
print (third_layer_weights)
print ('w23 = ', w23)
print (third_layer_bias)
print ('b3 = ', b3)

#use model.predict() function to print model predictions for data conditions
xarray= np.array(xdata)
print ('x01/20.2,  x02/14.5,   x03/308.0,  y3/32.4,  a3:')
test = []
for i in range(0,8):
    test = [[xarray[i][0], xarray[i][1], xarray[i][2]]]
    testarray = np.array(test)
    a3 = model.predict(testarray)
    print (xarray[i][0], xarray[i][1], xarray[i][2], df.y3[i], a3)
print('  ')
print ('x01,  x02,   x03,  y3,  a3*32.4:')
for i in range(0,8):
    test = [[xarray[i][0], xarray[i][1], xarray[i][2]]]
    testarray = np.array(test)
    a3 = model.predict(testarray)
    print (xarray[i][0]*20.2, xarray[i][1]*14.5, xarray[i][2]*308.0, df.y3[i
```

```
[[1.3305638 ]
 [0.36953682]
 [0.72552633]]
w01 =  1.3305638 w02 =  0.36953682 w03 =  0.72552633
[-0.29425886]
b1 =  [-0.29425886]
[[0.69765025]]
w12 =  0.69765025
[-0.14296538]
b2 =  [-0.14296538]
[[0.5929408]]
w23 =  0.5929408
[0.14315481]
b3 =  [0.14315481]
x01/20.2,  x02/14.5,   x03/308.0,  y3/32.4,  a3:
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step
0.9900990099009901 0.896551724137931 1.009090909090909 0.9564519613592172
[[0.92152303]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
0.9900990099009901 1.0 1.0 0.9937964877627233 [[0.9346082]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step
0.9900990099009901 1.0551724137931036 0.9935064935064936 0.9783648652819357
[[0.9410933]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
1.0 0.896551724137931 1.009090909090909 0.954291534211907 [[0.9269727]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step
0.9900990099009901 1.0 1.0 0.9999691367550383 [[0.9346082]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
1.0 1.0551724137931036 0.9935064935064936 0.9691058917934631 [[0.9465428]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
1.188118811881188 0.896551724137931 1.009090909090909 1.096571093484769 [[1.
0305148]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
1.7821782178217822 0.993103448275862 1.0003246753246755 1.4323631986667076
[[1.3696182]]

x01,  x02,   x03,  y3,  a3*32.4:
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step
20.0 13.0 310.8 30.989043548038634 [[29.857347]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step
20.0 14.5 308.0 32.19900620351223 [[30.281307]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step
20.0 15.3 306.0 31.699021635134713 [[30.491425]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step
20.2 13.0 310.8 30.919045708465788 [[30.033916]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step
20.0 14.5 308.0 32.39900003086324 [[30.281307]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
20.2 15.3 306.0 31.3990308941082 [[30.667988]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step
23.999999999999996 13.0 310.8 35.52890342890652 [[33.388683]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
36.0 14.4 308.1000000000001 46.40856763680132 [[44.37563]]
```

In [ ]:

In [20]:
```
"""
Single Log-Log Plot: Neural Network Predictions vs Actual Values
First Principles vs Keras Model - Using Actual Data

ME249 - Spray Cooling System Neural Network Project
V.P. Carey, Fall 2025
"""

import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# ACTUAL DATA FROM MODELS
# ============================================================

# First Principles Model Data
# Format: Tdbin, Twbin, qdot, Tdbout (actual), ypredicted
fp_data = [
    [20.0, 13.0, 310.8, 30.97, 31.187674466338166],
    [20.0, 14.5, 308.0, 32.3, 31.738492841534573],
    [20.0, 15.3, 306.0, 31.5, 32.01389557670518],
    [20.2, 13.0, 310.8, 30.91, 31.383400627268312],
    [20.0, 14.5, 308.0, 32.5, 31.738492841534573],
    [20.0, 15.3, 306.0, 31.4, 32.01389557670518],
    [24.0, 13.0, 310.8, 35.59, 35.1021976849411],
    [36.0, 14.4, 308.0, 46.41, 47.35672275614189]
]

# Keras Model Data
# Format: actual (y3), predicted (a3*32.4)
keras_data = [
    [30.989043548038634, 30.738674],
    [32.19900620351223, 31.221966],
    [31.699021635134713, 31.461542],
    [30.919045708465788, 30.924967],
    [32.39900003086324, 31.221966],
    [31.3990308941082, 31.647833],
    [35.52890342890652, 34.464474],
    [46.40856763680132, 46.089832]
]

# Extract data
fp_actual = [row[3] for row in fp_data]
fp_predicted = [row[4] for row in fp_data]

keras_actual = [row[0] for row in keras_data]
keras_predicted = [row[1] for row in keras_data]

# ============================================================
# PRINT DATA TABLE
# ============================================================

print("=" * 90)
print("FIRST PRINCIPLES MODEL")
print("=" * 90)
```

```
print(f"{'Index':<6} {'Actual (°C)':<15} {'Predicted (°C)':<18} {'Error (°C)
print("-" * 90)

fp_errors = []
for i in range(len(fp_actual)):
    error = fp_predicted[i] - fp_actual[i]
    error_pct = (error / fp_actual[i]) * 100
    fp_errors.append(error)
    print(f"{i:<6} {fp_actual[i]:<15.4f} {fp_predicted[i]:<18.4f} {error:<15

print("-" * 90)

print("\n" + "=" * 90)
print("KERAS MODEL")
print("=" * 90)
print(f"{'Index':<6} {'Actual (°C)':<15} {'Predicted (°C)':<18} {'Error (°C)
print("-" * 90)

keras_errors = []
for i in range(len(keras_actual)):
    error = keras_predicted[i] - keras_actual[i]
    error_pct = (error / keras_actual[i]) * 100
    keras_errors.append(error)
    print(f"{i:<6} {keras_actual[i]:<15.4f} {keras_predicted[i]:<18.4f} {err

print("-" * 90)

# ============================================================
# CREATE SINGLE LOG-LOG PLOT
# ============================================================

plt.figure(figsize=(11, 9))

# Plot both models on same plot
plt.loglog(fp_predicted, fp_actual, 'o', color='steelblue',
           markersize=14, label='First Principles', alpha=0.8,
           markeredgecolor='navy', markeredgewidth=2.5)

plt.loglog(keras_predicted, keras_actual, 's', color='coral',
           markersize=14, label='Keras Model', alpha=0.8,
           markeredgecolor='darkred', markeredgewidth=2.5)

# Perfect prediction line (y = x)
min_val = min(min(fp_predicted), min(keras_predicted), min(fp_actual), min(k
max_val = max(max(fp_predicted), max(keras_predicted), max(fp_actual), max(k
plt.loglog([min_val, max_val], [min_val, max_val], 'k--', linewidth=2.5,
           alpha=0.6, label='Perfect Prediction (y=x)', zorder=1)

# Labels and title
plt.xlabel('Predicted Y3 (Output Temperature, °C)', fontsize=13, fontweight=
plt.ylabel('Actual Y3 (Data Values, °C)', fontsize=13, fontweight='bold')
plt.title('Neural Network Model Comparison: Predicted vs Actual Output Tempe
          fontsize=14, fontweight='bold')

# Grid and legend
plt.grid(True, which='both', alpha=0.35, linestyle='--', linewidth=0.7)
```

```
plt.legend(fontsize=12, loc='best', framealpha=0.95, edgecolor='black', fanc

# Calculate statistics for both models
fp_mae = np.mean(np.abs(np.array(fp_errors)))
fp_rmse = np.sqrt(np.mean(np.array(fp_errors)**2))
fp_mape = np.mean(np.abs(np.array(fp_errors) / np.array(fp_actual))) * 100

keras_mae = np.mean(np.abs(np.array(keras_errors)))
keras_rmse = np.sqrt(np.mean(np.array(keras_errors)**2))
keras_mape = np.mean(np.abs(np.array(keras_errors) / np.array(keras_actual))

# Display statistics box
stats_text = (f'First Principles:\n'
              f'  MAE: {fp_mae:.4f}°C\n'
              f'  RMSE: {fp_rmse:.4f}°C\n'
              f'  MAPE: {fp_mape:.2f}%\n\n'
              f'Keras Model:\n'
              f'  MAE: {keras_mae:.4f}°C\n'
              f'  RMSE: {keras_rmse:.4f}°C\n'
              f'  MAPE: {keras_mape:.2f}%')

plt.text(0.05, 0.95, stats_text, transform=plt.gca().transAxes,
         fontsize=10, verticalalignment='top', family='monospace',
         bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.4,
                   edgecolor='black', linewidth=1.5, pad=0.8))

plt.tight_layout()
plt.savefig('loglog_plot_actual_data.png', dpi=300, bbox_inches='tight')
print("\n" + "=" * 90)
print("Plot saved to: loglog_plot_actual_data.png")
print("=" * 90)

# ============================================================
# DETAILED STATISTICS
# ============================================================

print("\n" + "=" * 90)
print("DETAILED PERFORMANCE STATISTICS")
print("=" * 90)

print(f"\n{'Metric':<40} {'First Principles':<25} {'Keras Model':<25}")
print("-" * 90)

print(f"{'Mean Absolute Error (MAE)':<40} {fp_mae:<25.6f}°C {keras_mae:<25.6
print(f"{'Root Mean Squared Error (RMSE)':<40} {fp_rmse:<25.6f}°C {keras_rms
print(f"{'Mean Absolute Percentage Error (MAPE)':<40} {fp_mape:<25.4f}% {ker

print(f"\n{'Min Prediction':<40} {min(fp_predicted):<25.4f}°C {min(keras_pre
print(f"{'Max Prediction':<40} {max(fp_predicted):<25.4f}°C {max(keras_predi
print(f"{'Mean Prediction':<40} {np.mean(fp_predicted):<25.4f}°C {np.mean(ke
print(f"{'Std Dev Prediction':<40} {np.std(fp_predicted):<25.4f}°C {np.std(k

print(f"\n{'Min Actual Data':<40} {min(fp_actual):<25.4f}°C {min(keras_actua
print(f"{'Max Actual Data':<40} {max(fp_actual):<25.4f}°C {max(keras_actual)
print(f"{'Mean Actual Data':<40} {np.mean(fp_actual):<25.4f}°C {np.mean(kera
print(f"{'Std Dev Actual Data':<40} {np.std(fp_actual):<25.4f}°C {np.std(ker
```

```
print("\n" + "=" * 90)

# Calculate R-squared (coefficient of determination)
fp_ss_res = np.sum(np.array(fp_errors)**2)
fp_ss_tot = np.sum((np.array(fp_actual) - np.mean(fp_actual))**2)
fp_r_squared = 1 - (fp_ss_res / fp_ss_tot)

keras_ss_res = np.sum(np.array(keras_errors)**2)
keras_ss_tot = np.sum((np.array(keras_actual) - np.mean(keras_actual))**2)
keras_r_squared = 1 - (keras_ss_res / keras_ss_tot)

print(f"\nR-squared (R²) - Goodness of Fit:")
print(f"  First Principles: {fp_r_squared:.6f}")
print(f"  Keras Model:      {keras_r_squared:.6f}")

print("\n" + "=" * 90)

plt.show()
```

```
================================================================================
==============
FIRST PRINCIPLES MODEL
================================================================================
==============
Index  Actual (°C)     Predicted (°C)     Error (°C)      Error (%)
--------------------------------------------------------------------------------
--------------
0      30.9700         31.1877            0.2177          0.70      %
1      32.3000         31.7385            -0.5615         -1.74     %
2      31.5000         32.0139            0.5139          1.63      %
3      30.9100         31.3834            0.4734          1.53      %
4      32.5000         31.7385            -0.7615         -2.34     %
5      31.4000         32.0139            0.6139          1.96      %
6      35.5900         35.1022            -0.4878         -1.37     %
7      46.4100         47.3567            0.9467          2.04      %
--------------------------------------------------------------------------------
--------------

================================================================================
==============
KERAS MODEL
================================================================================
==============
Index  Actual (°C)     Predicted (°C)     Error (°C)      Error (%)
--------------------------------------------------------------------------------
--------------
0      30.9890         30.7387            -0.2504         -0.81     %
1      32.1990         31.2220            -0.9770         -3.03     %
2      31.6990         31.4615            -0.2375         -0.75     %
3      30.9190         30.9250            0.0059          0.02      %
4      32.3990         31.2220            -1.1770         -3.63     %
5      31.3990         31.6478            0.2488          0.79      %
6      35.5289         34.4645            -1.0644         -3.00     %
7      46.4086         46.0898            -0.3187         -0.69     %
--------------------------------------------------------------------------------
--------------

================================================================================
==============
Plot saved to: loglog_plot_actual_data.png
================================================================================
==============

================================================================================
==============
DETAILED PERFORMANCE STATISTICS
================================================================================
==============

Metric                                   First Principles          Keras Mod
el
--------------------------------------------------------------------------------
--------------
Mean Absolute Error (MAE)                0.572051                  °C 0.53497
6                °C
```

```
Root Mean Squared Error (RMSE)           0.606410                °C 0.68516
8                °C
Mean Absolute Percentage Error (MAPE)    1.6641                  % 1.5898
%

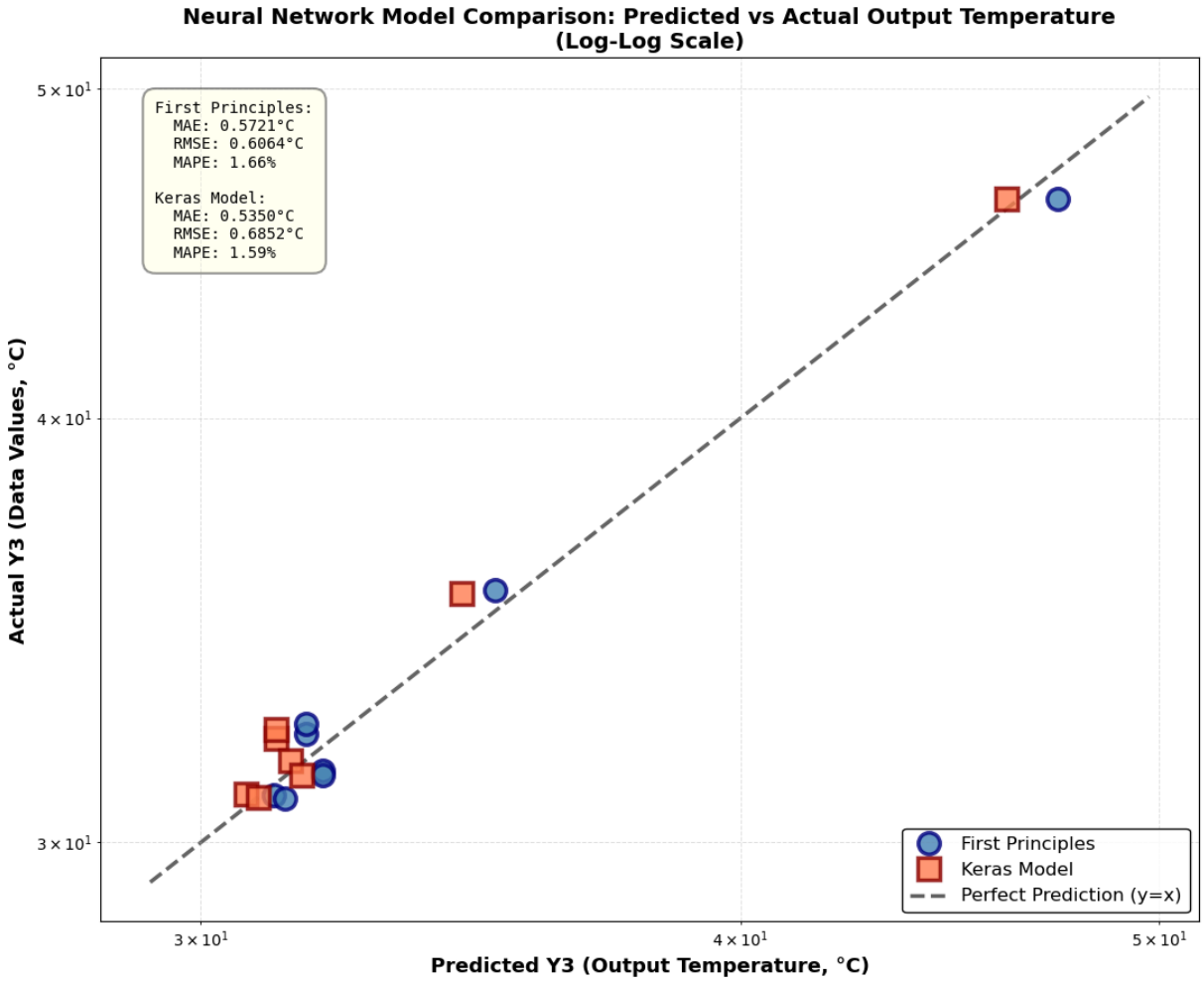
Min Prediction                           31.1877                 °C 30.7387
°C
Max Prediction                           47.3567                 °C 46.0898
°C
Mean Prediction                          34.0668                 °C 33.4714
°C
Std Dev Prediction                       5.1533                  °C 4.8945
°C

Min Actual Data                          30.9100                 °C 30.9190
°C
Max Actual Data                          46.4100                 °C 46.4086
°C
Mean Actual Data                         33.9475                 °C 33.9427
°C
Std Dev Actual Data                      4.9167                  °C 4.9080
°C

================================================================================
==============

R-squared (R²) - Goodness of Fit:
  First Principles: 0.984788
  Keras Model:      0.980511

================================================================================
==============
```


Neural Network Model Comparison: Predicted vs Actual Output Temperature
(Log-Log Scale)
First Principles:
MAE: 0.5721°C
RMSE: 0.6064°C
MAPE: 1.66%
Keras Model:
MAE: 0.5350°C
RMSE: 0.6852°C
MAPE: 1.59%
First Principles
Keras Model
Perfect Prediction (y=x)
Actual Y3 (Data Values, °C)
Predicted Y3 (Output Temperature, °C)
5 × 10^1
4 × 10^1
3 × 10^1

In [ ]:

In [27]:
```
%reset -f
'''>>>>> start CodeP2.3F25
    V.P. Carey ME249, Fall 2025

Intro to Neural Network Modeling
Data arrays for hybrid solar/fossil-fuel gas turbine power system'''
import numpy as np

#create input data array, normalizing input temp
#T1(K), gamma, , qsol(kW):
xdata = []
xdata =  [[ 318.0 , 0.0 , 500.0 ], [ 318.0 , 0.0 , 1000.0 ]]
xdata.append([ 318.0 , 0.0 , 1500.0 ])
xdata.append([ 318.0 , 0.0 , 2000.0 ])
xdata.append([ 318.0 , 0.0 , 2500.0 ])
xdata.append([ 318.0 , 0.25 , 500.0 ])
xdata.append([ 318.0 , 0.25 , 1000.0 ])
xdata.append([ 318.0 , 0.25 , 1500.0 ])
xdata.append([ 318.0 , 0.25 , 2000.0 ])
xdata.append([ 318.0 , 0.25 , 2500.0 ])
xdata.append([ 318.0 , 0.5 , 500.0 ])
xdata.append([ 318.0 , 0.5 , 1000.0 ])
xdata.append([ 318.0 , 0.5 , 1500.0 ])
xdata.append([ 318.0 , 0.5 , 2000.0 ])
xdata.append([ 318.0 , 0.5 , 2501.0 ])

xdata.append([ 303.0 , 0.0 , 500.0 ])
xdata.append([ 303.0 , 0.0 , 1000.0 ])
xdata.append([ 303.0 , 0.0 , 1500.0 ])
xdata.append([ 303.0 , 0.0 , 2000.0 ])
xdata.append([ 303.0 , 0.0 , 2500.0 ])
xdata.append([ 303.0 , 0.25 , 500.0 ])
xdata.append([ 303.0 , 0.25 , 1000.0 ])
xdata.append([ 303.0 , 0.25 , 1500.0 ])
xdata.append([ 303.0 , 0.25 , 2000.0 ])
xdata.append([ 303.0 , 0.25 , 2500.0 ])
xdata.append([ 303.0 , 0.5 , 500.0 ])
xdata.append([ 303.0 , 0.5 , 1000.0 ])
xdata.append([ 303.0 , 0.5 , 1500.0 ])
xdata.append([ 303.0 , 0.5 , 2000.0 ])
xdata.append([ 303.0 , 0.5 , 2500.0 ])

xdata.append([ 288.0 , 0.0 , 500.0 ])
xdata.append([ 288.0 , 0.0 , 1000.0 ])
xdata.append([ 288.0 , 0.0 , 1500.0 ])
xdata.append([ 288.0 , 0.0 , 2000.0 ])
xdata.append([ 288.0 , 0.0 , 2500.0 ])
xdata.append([ 288.0 , 0.25 , 500.0 ])
xdata.append([ 288.0 , 0.25 , 1000.0 ])
xdata.append([ 288.0 , 0.25 , 1500.0 ])
xdata.append([ 288.0 , 0.25 , 2000.0 ])
xdata.append([ 288.0 , 0.25 , 2500.0 ])
xdata.append([ 288.0 , 0.51 , 500.0 ])
xdata.append([ 288.0 , 0.5 , 1000.0 ])
```

```
xdata.append([ 288.0 , 0.5 , 1500.0 ])
xdata.append([ 288.0 , 0.5 , 2000.0 ])
xdata.append([ 288.0 , 0.5 , 2500.0 ])

xdata.append([ 268.0 , 0.0 , 500.0 ])
xdata.append([ 268.0 , 0.0 , 1000.0 ])
xdata.append([ 268.0 , 0.0 , 1500.0 ])
xdata.append([ 268.0 , 0.0 , 2000.0 ])
xdata.append([ 268.0 , 0.0 , 2500.0 ])
xdata.append([ 268.0 , 0.25 , 500.0 ])
xdata.append([ 268.0 , 0.25 , 1000.0 ])
xdata.append([ 268.0 , 0.25 , 1500.0 ])
xdata.append([ 268.0 , 0.25 , 2000.0 ])
xdata.append([ 268.0 , 0.25 , 2500.0 ])
xdata.append([ 268.0 , 0.5 , 500.0 ])
xdata.append([ 268.0 , 0.5 , 1000.0 ])
xdata.append([ 268.0 , 0.5 , 1500.0 ])
xdata.append([ 268.0 , 0.5 , 2000.0 ])
xdata.append([ 268.0 , 0.5 , 2500.0 ])



#outputs
#alpha = air/fuel ratio , cycle efficiency:
ydata =  [[ 35.1316 , 0.3808 ],[ 40.3764 , 0.38686 ]]
ydata.append([ 47.4620 , 0.3930 ])
ydata.append([ 57.5639 , 0.39949 ])
ydata.append([ 73.1286 , 0.40612 ])
ydata.append([ 49.1110 , 0.4023 ])
ydata.append([ 56.4428 , 0.40605 ])
ydata.append([ 66.3479 , 0.4098 ])
ydata.append([ 80.4695 , 0.4133])
ydata.append([ 102.2276 , 0.4175 ])
ydata.append([ 63.0904 , 0.41540 ])
ydata.append([ 72.5092 , 0.4175 ])
ydata.append([ 85.2338, 0.4197 ])
ydata.append([ 103.3750 , 0.42192 ])
ydata.append([ 131.3266 , 0.4242 ])

ydata.append([ 34.273 , 0.3952 ])
ydata.append([ 38.99026 , 0.4012 ])
ydata.append([ 45.2133, 0.4073 ])
ydata.append([ 53.8000 , 0.4136 ])
ydata.append([ 66.4130 , 0.4201 ])
ydata.append([ 47.922 , 0.4178 ])
ydata.append([ 54.518 , 0.4215 ])
ydata.append([ 63.220 , 0.4252 ])
ydata.append([ 75.226 , 0.4290 ])
ydata.append([ 92.862 , 0.4329 ])
ydata.append([ 61.572 , 0.4315 ])
ydata.append([ 70.0468 , 0.43373 ])
ydata.append([ 81.226 , 0.43597 ])
ydata.append([ 96.653 , 0.4382 ])
ydata.append([ 119.3124 , 0.44045 ])

ydata.append([ 33.4521 , 0.40913 ])
```

```
ydata.append([ 37.6911, 0.4150 ])
ydata.append([ 43.1602 , 0.4209 ])
ydata.append([ 50.4858 , 0.4271 ])
ydata.append([ 60.8067 , 0.4334 ])
ydata.append([ 46.7865 , 0.4328 ])
ydata.append([ 52.7151 , 0.43646 ])
ydata.append([ 60.36425 , 0.44016 ])
ydata.append([ 70.6099 , 0.443926 ])
ydata.append([ 85.0447 , 0.4477 ])
ydata.append([ 60.1208 , 0.44721 ])
ydata.append([ 67.7391 , 0.44940 ])
ydata.append([ 77.56830 , 0.4516 ])
ydata.append([ 90.73410 , 0.4538 ])
ydata.append([ 109.2828 , 0.4560 ])

ydata.append([ 32.4123 , 0.42694 ])
ydata.append([ 36.0807 , 0.4325 ])
ydata.append([ 40.6854 , 0.4383 ])
ydata.append([ 46.6374 , 0.4442 ])
ydata.append([ 54.6293 , 0.4503 ])
ydata.append([ 45.3472 , 0.4519 ])
ydata.append([ 50.4796 , 0.4555 ])
ydata.append([ 56.9219 , 0.4591 ])
ydata.append([ 65.2492 , 0.4628 ])
ydata.append([ 76.4304 , 0.4665 ])
ydata.append([ 58.2822 , 0.4672 ])
ydata.append([ 64.8785 , 0.4693 ])
ydata.append([ 73.1584 , 0.4715 ])
ydata.append([ 83.8610 , 0.4738 ])
ydata.append([ 98.2316 , 0.4760 ])

print(xdata)
print(ydata)

# merged = np.hstack((xdata, ydata), dtype=float)
# print(merged)
```

```
[[318.0, 0.0, 500.0], [318.0, 0.0, 1000.0], [318.0, 0.0, 1500.0], [318.0, 0.
0, 2000.0], [318.0, 0.0, 2500.0], [318.0, 0.25, 500.0], [318.0, 0.25, 1000.
0], [318.0, 0.25, 1500.0], [318.0, 0.25, 2000.0], [318.0, 0.25, 2500.0], [31
8.0, 0.5, 500.0], [318.0, 0.5, 1000.0], [318.0, 0.5, 1500.0], [318.0, 0.5, 2
000.0], [318.0, 0.5, 2501.0], [303.0, 0.0, 500.0], [303.0, 0.0, 1000.0], [30
3.0, 0.0, 1500.0], [303.0, 0.0, 2000.0], [303.0, 0.0, 2500.0], [303.0, 0.25,
500.0], [303.0, 0.25, 1000.0], [303.0, 0.25, 1500.0], [303.0, 0.25, 2000.0],
[303.0, 0.25, 2500.0], [303.0, 0.5, 500.0], [303.0, 0.5, 1000.0], [303.0, 0.
5, 1500.0], [303.0, 0.5, 2000.0], [303.0, 0.5, 2500.0], [288.0, 0.0, 500.0],
[288.0, 0.0, 1000.0], [288.0, 0.0, 1500.0], [288.0, 0.0, 2000.0], [288.0, 0.
0, 2500.0], [288.0, 0.25, 500.0], [288.0, 0.25, 1000.0], [288.0, 0.25, 1500.
0], [288.0, 0.25, 2000.0], [288.0, 0.25, 2500.0], [288.0, 0.51, 500.0], [28
8.0, 0.5, 1000.0], [288.0, 0.5, 1500.0], [288.0, 0.5, 2000.0], [288.0, 0.5,
2500.0], [268.0, 0.0, 500.0], [268.0, 0.0, 1000.0], [268.0, 0.0, 1500.0], [2
68.0, 0.0, 2000.0], [268.0, 0.0, 2500.0], [268.0, 0.25, 500.0], [268.0, 0.2
5, 1000.0], [268.0, 0.25, 1500.0], [268.0, 0.25, 2000.0], [268.0, 0.25, 250
0.0], [268.0, 0.5, 500.0], [268.0, 0.5, 1000.0], [268.0, 0.5, 1500.0], [268.
0, 0.5, 2000.0], [268.0, 0.5, 2500.0]]
[[35.1316, 0.3808], [40.3764, 0.38686], [47.462, 0.393], [57.5639, 0.39949],
[73.1286, 0.40612], [49.111, 0.4023], [56.4428, 0.40605], [66.3479, 0.4098],
[80.4695, 0.4133], [102.2276, 0.4175], [63.0904, 0.4154], [72.5092, 0.4175],
[85.2338, 0.4197], [103.375, 0.42192], [131.3266, 0.4242], [34.273, 0.3952],
[38.99026, 0.4012], [45.2133, 0.4073], [53.8, 0.4136], [66.413, 0.4201], [4
7.922, 0.4178], [54.518, 0.4215], [63.22, 0.4252], [75.226, 0.429], [92.862,
0.4329], [61.572, 0.4315], [70.0468, 0.43373], [81.226, 0.43597], [96.653,
0.4382], [119.3124, 0.44045], [33.4521, 0.40913], [37.6911, 0.415], [43.160
2, 0.4209], [50.4858, 0.4271], [60.8067, 0.4334], [46.7865, 0.4328], [52.715
1, 0.43646], [60.36425, 0.44016], [70.6099, 0.443926], [85.0447, 0.4477], [6
0.1208, 0.44721], [67.7391, 0.4494], [77.5683, 0.4516], [90.7341, 0.4538],
[109.2828, 0.456], [32.4123, 0.42694], [36.0807, 0.4325], [40.6854, 0.4383],
[46.6374, 0.4442], [54.6293, 0.4503], [45.3472, 0.4519], [50.4796, 0.4555],
[56.9219, 0.4591], [65.2492, 0.4628], [76.4304, 0.4665], [58.2822, 0.4672],
[64.8785, 0.4693], [73.1584, 0.4715], [83.861, 0.4738], [98.2316, 0.476]]
```

In [28]:

```
xdata = np.array(xdata)
ydata = np.array(ydata)

# Calculate medians once
medians_x = np.array([np.median(xdata[:, 0]),
                      np.median(xdata[:, 1]),
                      np.median(xdata[:, 2])])
medians_y = np.array([np.median(ydata[:, 0]),
                      np.median(ydata[:, 1])])

# Normalize all at once
xdata = xdata / medians_x
ydata = ydata / medians_y

# Print if needed
print(medians_x[0], medians_x[1], medians_x[2], medians_y[0], medians_y[1])

# print(xdata)
# print(ydata)
```

```
295.5 0.25 1500.0 61.189350000000005 0.432
```

In [ ]:

Cell 1

In [1]:

```
'''>>>>> start CodeP2.4F25
    V.P. Carey ME249, Fall 2025

Intro to Neural Network Modeling
Keras model for hybrid solar/fossil-fuel gas turbine power system'''
%reset -f
#import useful packages
import keras
import pandas as pd
from keras.models import Sequential
import numpy as np
import keras.backend as kb
import tensorflow as tf
#the follwoing 2 lines are only needed for Mac OS machines
import os
os.environ['KMP_DUPLICATE_LIB_OK']='True'

#create input data array

# #T1(K), gamma, , qsol(kW):
#convert to:
# meadian values of input variables
Tmed = 293.
gamed = 0.25
qsmed = 1250.

#T1(K), gamma, , qsol(kW):
xdata = []
ND = 60
xdata =  [[ 318.0 , 0.0 , 500.0 ], [ 318.0 , 0.0 , 1000.0 ]]
xdata.append([ 318.0  , 0.0 , 1500.0])
xdata.append([ 318.0  , 0.0 , 2000.0 ])
xdata.append([ 318.0  , 0.0 , 2500.0 ])
#appended dataset
xdata.append ([ 318.0 , 0.0 , 500.0 ])
xdata.append ([ 318.0 , 0.0 , 1000.0 ])
xdata.append([ 318.0 , 0.0 , 1500.0 ])
xdata.append([ 318.0 , 0.0 , 2000.0 ])
xdata.append([ 318.0 , 0.0 , 2500.0 ])
xdata.append([ 318.0 , 0.25 , 500.0 ])
xdata.append([ 318.0 , 0.25 , 1000.0 ])
xdata.append([ 318.0 , 0.25 , 1500.0 ])
xdata.append([ 318.0 , 0.25 , 2000.0 ])
xdata.append([ 318.0 , 0.25 , 2500.0 ])
xdata.append([ 318.0 , 0.5 , 500.0 ])
xdata.append([ 318.0 , 0.5 , 1000.0 ])
xdata.append([ 318.0 , 0.5 , 1500.0 ])
xdata.append([ 318.0 , 0.5 , 2000.0 ])
xdata.append([ 318.0 , 0.5 , 2501.0 ])

xdata.append([ 303.0 , 0.0 , 500.0 ])
```

```
xdata.append([ 303.0 , 0.0 , 1000.0 ])
xdata.append([ 303.0 , 0.0 , 1500.0 ])
xdata.append([ 303.0 , 0.0 , 2000.0 ])
xdata.append([ 303.0 , 0.0 , 2500.0 ])
xdata.append([ 303.0 , 0.25 , 500.0 ])
xdata.append([ 303.0 , 0.25 , 1000.0 ])
xdata.append([ 303.0 , 0.25 , 1500.0 ])
xdata.append([ 303.0 , 0.25 , 2000.0 ])
xdata.append([ 303.0 , 0.25 , 2500.0 ])
xdata.append([ 303.0 , 0.5 , 500.0 ])
xdata.append([ 303.0 , 0.5 , 1000.0 ])
xdata.append([ 303.0 , 0.5 , 1500.0 ])
xdata.append([ 303.0 , 0.5 , 2000.0 ])
xdata.append([ 303.0 , 0.5 , 2500.0 ])

xdata.append([ 288.0 , 0.0 , 500.0 ])
xdata.append([ 288.0 , 0.0 , 1000.0 ])
xdata.append([ 288.0 , 0.0 , 1500.0 ])
xdata.append([ 288.0 , 0.0 , 2000.0 ])
xdata.append([ 288.0 , 0.0 , 2500.0 ])
xdata.append([ 288.0 , 0.25 , 500.0 ])
xdata.append([ 288.0 , 0.25 , 1000.0 ])
xdata.append([ 288.0 , 0.25 , 1500.0 ])
xdata.append([ 288.0 , 0.25 , 2000.0 ])
xdata.append([ 288.0 , 0.25 , 2500.0 ])
xdata.append([ 288.0 , 0.51 , 500.0 ])
xdata.append([ 288.0 , 0.5 , 1000.0 ])
xdata.append([ 288.0 , 0.5 , 1500.0 ])
xdata.append([ 288.0 , 0.5 , 2000.0 ])
xdata.append([ 288.0 , 0.5 , 2500.0 ])

xdata.append([ 268.0 , 0.0 , 500.0 ])
xdata.append([ 268.0 , 0.0 , 1000.0 ])
xdata.append([ 268.0 , 0.0 , 1500.0 ])
xdata.append([ 268.0 , 0.0 , 2000.0 ])
xdata.append([ 268.0 , 0.0 , 2500.0 ])
xdata.append([ 268.0 , 0.25 , 500.0 ])
xdata.append([ 268.0 , 0.25 , 1000.0 ])
xdata.append([ 268.0 , 0.25 , 1500.0 ])
xdata.append([ 268.0 , 0.25 , 2000.0 ])
xdata.append([ 268.0 , 0.25 , 2500.0 ])
xdata.append([ 268.0 , 0.5 , 500.0 ])
xdata.append([ 268.0 , 0.5 , 1000.0 ])
xdata.append([ 268.0 , 0.5 , 1500.0 ])
xdata.append([ 268.0 , 0.5 , 2000.0 ])
xdata.append([ 268.0 , 0.5 , 2500.0 ])
xarray= np.array(xdata)

# add the rest here
new_Tmed  = np.median(xarray[:,0])
new_gamed = np.median(xarray[:,1])
new_qsmed = np.median(xarray[:,2])

input = xarray/[new_Tmed,new_gamed,new_qsmed]

# print (xdata)
```

```
# print (xarray)

#alpha = air/fuel ratio , cycle efficiency:
#convert to:
# meadian values of output variables
almed = 60.
efmed = 0.4
# alpha, effsys

ydata = []
ydata =  [[ 35.1316 , 0.3808 ], [ 40.3764 , 0.38686]]
ydata.append([ 47.4620 , 0.3930])
ydata.append([ 57.5639 , 0.39949])
ydata.append([ 73.1286 , 0.40612 ])
#appended dataset
ydata.append([ 35.1316 , 0.3808 ])
ydata.append([ 40.3764 , 0.38686 ])
ydata.append([ 47.4620 , 0.3930 ])
ydata.append([ 57.5639 , 0.39949 ])
ydata.append([ 73.1286 , 0.40612 ])
ydata.append([ 49.1110 , 0.4023 ])
ydata.append([ 56.4428 , 0.40605 ])
ydata.append([ 66.3479 , 0.4098 ])
ydata.append([ 80.4695 , 0.4133])
ydata.append([ 102.2276 , 0.4175 ])
ydata.append([ 63.0904 , 0.41540 ])
ydata.append([ 72.5092 , 0.4175 ])
ydata.append([ 85.2338, 0.4197 ])
ydata.append([ 103.3750 , 0.42192 ])
ydata.append([ 131.3266 , 0.4242 ])

ydata.append([ 34.273 , 0.3952 ])
ydata.append([ 38.99026 , 0.4012 ])
ydata.append([ 45.2133, 0.4073 ])
ydata.append([ 53.8000 , 0.4136 ])
ydata.append([ 66.4130 , 0.4201 ])
ydata.append([ 47.922 , 0.4178 ])
ydata.append([ 54.518 , 0.4215 ])
ydata.append([ 63.220 , 0.4252 ])
ydata.append([ 75.226 , 0.4290 ])
ydata.append([ 92.862 , 0.4329 ])
ydata.append([ 61.572 , 0.4315 ])
ydata.append([ 70.0468 , 0.43373 ])
ydata.append([ 81.226 , 0.43597 ])
ydata.append([ 96.653 , 0.4382 ])
ydata.append([ 119.3124 , 0.44045 ])

ydata.append([ 33.4521 , 0.40913 ])
ydata.append([ 37.6911, 0.4150 ])
ydata.append([ 43.1602 , 0.4209 ])
ydata.append([ 50.4858 , 0.4271 ])
ydata.append([ 60.8067 , 0.4334 ])
ydata.append([ 46.7865 , 0.4328 ])
ydata.append([ 52.7151 , 0.43646 ])
ydata.append([ 60.36425 , 0.44016 ])
ydata.append([ 70.6099 , 0.443926 ])
```

```
ydata.append([ 85.0447 , 0.4477 ])
ydata.append([ 60.1208 , 0.44721 ])
ydata.append([ 67.7391 , 0.44940 ])
ydata.append([ 77.56830 , 0.4516 ])
ydata.append([ 90.73410 , 0.4538 ])
ydata.append([ 109.2828 , 0.4560 ])

ydata.append([ 32.4123 , 0.42694 ])
ydata.append([ 36.0807 , 0.4325 ])
ydata.append([ 40.6854 , 0.4383 ])
ydata.append([ 46.6374 , 0.4442 ])
ydata.append([ 54.6293 , 0.4503 ])
ydata.append([ 45.3472 , 0.4519 ])
ydata.append([ 50.4796 , 0.4555 ])
ydata.append([ 56.9219 , 0.4591 ])
ydata.append([ 65.2492 , 0.4628 ])
ydata.append([ 76.4304 , 0.4665 ])
ydata.append([ 58.2822 , 0.4672 ])
ydata.append([ 64.8785 , 0.4693 ])
ydata.append([ 73.1584 , 0.4715 ])
ydata.append([ 83.8610 , 0.4738 ])
ydata.append([ 98.2316 , 0.4760 ])
yarray= np.array(ydata)

new_almed = np.median(yarray[:,0])
new_efmed = np.median(yarray[:,1])
output = yarray / [new_almed, new_efmed]

#print (ydata)
#print (yarray)

data_inputs = np.array(input)
data_outputs = np.array(output)

print(f'==== Input ====')
print(data_inputs)
print(f'==== Output ====')
print(data_outputs)
```

```
==== Input ====
[[1.04950495 0.         0.33333333]
 [1.04950495 0.         0.66666667]
 [1.04950495 0.         1.        ]
 [1.04950495 0.         1.33333333]
 [1.04950495 0.         1.66666667]
 [1.04950495 0.         0.33333333]
 [1.04950495 0.         0.66666667]
 [1.04950495 0.         1.        ]
 [1.04950495 0.         1.33333333]
 [1.04950495 0.         1.66666667]
 [1.04950495 1.         0.33333333]
 [1.04950495 1.         0.66666667]
 [1.04950495 1.         1.        ]
 [1.04950495 1.         1.33333333]
 [1.04950495 1.         1.66666667]
 [1.04950495 2.         0.33333333]
 [1.04950495 2.         0.66666667]
 [1.04950495 2.         1.        ]
 [1.04950495 2.         1.33333333]
 [1.04950495 2.         1.66733333]
 [1.         0.         0.33333333]
 [1.         0.         0.66666667]
 [1.         0.         1.        ]
 [1.         0.         1.33333333]
 [1.         0.         1.66666667]
 [1.         1.         0.33333333]
 [1.         1.         0.66666667]
 [1.         1.         1.        ]
 [1.         1.         1.33333333]
 [1.         1.         1.66666667]
 [1.         2.         0.33333333]
 [1.         2.         0.66666667]
 [1.         2.         1.        ]
 [1.         2.         1.33333333]
 [1.         2.         1.66666667]
 [0.95049505 0.         0.33333333]
 [0.95049505 0.         0.66666667]
 [0.95049505 0.         1.        ]
 [0.95049505 0.         1.33333333]
 [0.95049505 0.         1.66666667]
 [0.95049505 1.         0.33333333]
 [0.95049505 1.         0.66666667]
 [0.95049505 1.         1.        ]
 [0.95049505 1.         1.33333333]
 [0.95049505 1.         1.66666667]
 [0.95049505 2.04       0.33333333]
 [0.95049505 2.         0.66666667]
 [0.95049505 2.         1.        ]
 [0.95049505 2.         1.33333333]
 [0.95049505 2.         1.66666667]
 [0.88448845 0.         0.33333333]
 [0.88448845 0.         0.66666667]
 [0.88448845 0.         1.        ]
 [0.88448845 0.         1.33333333]
 [0.88448845 0.         1.66666667]
```

```
 [0.88448845 1.         0.33333333]
 [0.88448845 1.         0.66666667]
 [0.88448845 1.         1.        ]
 [0.88448845 1.         1.33333333]
 [0.88448845 1.         1.66666667]
 [0.88448845 2.         0.33333333]
 [0.88448845 2.         0.66666667]
 [0.88448845 2.         1.        ]
 [0.88448845 2.         1.33333333]
 [0.88448845 2.         1.66666667]]
==== Output ====
[[0.58199348 0.89159447]
 [0.66887934 0.90578319]
 [0.78626008 0.92015921]
 [0.95360913 0.93535472]
 [1.21145546 0.95087801]
 [0.58199348 0.89159447]
 [0.66887934 0.90578319]
 [0.78626008 0.92015921]
 [0.95360913 0.93535472]
 [1.21145546 0.95087801]
 [0.81357757 0.94193397]
 [0.93503688 0.95071412]
 [1.09912572 0.95949426]
 [1.33306551 0.96768907]
 [1.6935123  0.97752283]
 [1.04516166 0.97260595]
 [1.20119442 0.97752283]
 [1.41199137 0.98267385]
 [1.71252024 0.98787169]
 [2.17556915 0.99321002]
 [0.56776983 0.92531023]
 [0.64591642 0.93935846]
 [0.7490079  0.95364083]
 [0.891256   0.96839148]
 [1.10020418 0.9836104 ]
 [0.79388048 0.97822524]
 [0.90315046 0.98688832]
 [1.04730863 0.99555139]
 [1.24620119 1.00444861]
 [1.53836087 1.01357996]
 [1.0200077  1.01030204]
 [1.16040206 1.0155233 ]
 [1.34559777 1.02076797]
 [1.60116294 1.02598923]
 [1.97654075 1.03125732]
 [0.55417072 0.95792554]
 [0.62439441 0.9716694 ]
 [0.71499604 0.98548349]
 [0.83635264 1.        ]
 [1.00732967 1.01475064]
 [0.77506968 1.01334582]
 [0.87328344 1.02191524]
 [1.         1.03057832]
 [1.16973043 1.03939593]
 [1.40885872 1.04823226]
```

```
[0.99596698 1.04708499]
[1.12217248 1.0522126 ]
[1.28500396 1.05736362]
[1.50310987 1.06251463]
[1.81038943 1.06766565]
[0.53694529 0.99962538]
[0.59771636 1.01264341]
[0.67399827 1.02622337]
[0.77259968 1.04003746]
[0.90499426 1.05431983]
[0.7512261  1.05806603]
[0.83624993 1.06649497]
[0.9429737  1.07492391]
[1.08092455 1.08358698]
[1.26615339 1.09225006]
[0.96550856 1.09388902]
[1.0747835  1.0988059 ]
[1.21194913 1.10395692]
[1.38924943 1.10934207]
[1.62731418 1.11449309]]
```

Cell 2

In [2]:

```
# define neural network model

#As seen below, we have created four dense layers.
#A dense layer is a layer in neural network that's fully connected.
#In other words, all the neurons in one layer are connected to all other neu
#In the first layer, we need to provide the input shape, which is 1 in our c
#The activation function we have chosen is elu, which stands for exponential

from keras import backend as K
#initialize weights with values between -0.2 and 1.2
initializer = keras.initializers.RandomUniform(minval= -0.9, maxval=0.9)

model = keras.Sequential([
    keras.layers.Dense(4, activation='relu', input_shape=[3],  kernel_initia
    keras.layers.Dense(8, activation='relu',  kernel_initializer=initializer
    keras.layers.Dense(4, activation='relu',  kernel_initializer=initializer

    keras.layers.Dense(2,  kernel_initializer=initializer)
  ])

'''in Task 2.2, add 3rd layer to network with 4 neurons and activation = K.r
#Print summary of model features
model.summary()
```

```
/global/home/users/kwyang0331/.conda/envs/wrf/lib/python3.10/site-packages/k
eras/src/layers/core/dense.py:95: UserWarning: Do not pass an `input_shape`/
`input_dim` argument to a layer. When using Sequential models, prefer using
an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Par
dense (Dense)	(None, 4)	
dense_1 (Dense)	(None, 8)	
dense_2 (Dense)	(None, 4)	
dense_3 (Dense)	(None, 2)	

Total params: 102 (408.00 B)
Trainable params: 102 (408.00 B)
Non-trainable params: 0 (0.00 B)

Cell 3

In [3]:
```
#We're using RMSprop as our optimizer here. RMSprop stands for Root Mean Squ
#It's one of the most popular gradient descent optimization algorithms for c
#RMSprop is an optimizer that's reliable and fast.
#We're compiling the mode using the model.compile function. The loss functic
#is mean squared error. After the compilation of the model, we'll use the fi
#Number of epochs can be varied.

#from tf.keras import optimizers. Argument to RMSprop is learning parameter.
rms = keras.optimizers.RMSprop(0.001)
model.compile(loss='mean_absolute_error',optimizer=rms)
```

Cell 4

In [68]:
```
#After the compilation of the model, we'll use the fit method with 500 epoch
#I started with epochs value of 100 and then tested the model after training
#The prediction was not that good. Then I modified the number of epochs to 2
#Accuracy had improved slightly, but figured I'd give it one more try. Final
#I found acceptable prediction accuracy.

#The fit method takes three parameters; namely, x, y, and number of epochs.
#During model training, if all the batches of data are seen by the model onc
#we say that one epoch has been completed.

# Add an early stopping callback
es = keras.callbacks.EarlyStopping(
    monitor='loss',
    mode='min',
    patience = 150,
    restore_best_weights = True,
    verbose=1)
# Add a checkpoint where loss is minimum, and save that model
mc = keras.callbacks.ModelCheckpoint('best_model.keras', monitor='loss',
                     mode='min',  verbose=1, save_best_only=True)

historyData = model.fit(data_inputs,data_outputs,epochs=600,callbacks=[es, m
```

```
loss_hist = historyData.history['loss']
#The above line will return a dictionary, access it's info like this:
best_epoch = np.argmin(historyData.history['loss']) + 1
print ('best epoch = ', best_epoch)
print('smallest loss =', np.min(loss_hist))

predictions = model.predict(data_inputs)
```

```
Epoch 1/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0518
Epoch 1: loss improved from None to 0.04979, saving model to best_model.kera
s
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - loss: 0.0498
Epoch 2/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - loss: 0.0498
Epoch 2: loss did not improve from 0.04979
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0510
Epoch 3/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0613
Epoch 3: loss did not improve from 0.04979
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0507
Epoch 4/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0558
Epoch 4: loss did not improve from 0.04979
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0524
Epoch 5/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0523
Epoch 5: loss did not improve from 0.04979
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0504
Epoch 6/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0443
Epoch 6: loss did not improve from 0.04979
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0505
Epoch 7/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0399
Epoch 7: loss did not improve from 0.04979
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0509
Epoch 8/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0559
Epoch 8: loss did not improve from 0.04979
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0528
Epoch 9/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0512
Epoch 9: loss did not improve from 0.04979
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0510
Epoch 10/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0529
Epoch 10: loss improved from 0.04979 to 0.04954, saving model to best_model.
keras
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0495
Epoch 11/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0454
Epoch 11: loss did not improve from 0.04954
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0502
Epoch 12/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0594
Epoch 12: loss did not improve from 0.04954
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0512
Epoch 13/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0655
Epoch 13: loss did not improve from 0.04954
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0511
Epoch 14/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0588
```

```
Epoch 14: loss did not improve from 0.04954
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0526
Epoch 15/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0594
Epoch 15: loss did not improve from 0.04954
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0534
Epoch 16/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0485
Epoch 16: loss improved from 0.04954 to 0.04941, saving model to best_model.
keras
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0494
Epoch 17/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0557
Epoch 17: loss did not improve from 0.04941
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0515
Epoch 18/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0492
Epoch 18: loss did not improve from 0.04941
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0504
Epoch 19/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0430
Epoch 19: loss did not improve from 0.04941
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0520
Epoch 20/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0603
Epoch 20: loss did not improve from 0.04941
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0527
Epoch 21/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0473
Epoch 21: loss did not improve from 0.04941
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0505
Epoch 22/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0498
Epoch 22: loss did not improve from 0.04941
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0534
Epoch 23/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0484
Epoch 23: loss improved from 0.04941 to 0.04936, saving model to best_model.
keras
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0494
Epoch 24/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0467
Epoch 24: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0515
Epoch 25/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0524
Epoch 25: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0497
Epoch 26/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0502
Epoch 26: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0500
Epoch 27/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0502
Epoch 27: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0514
```

```
Epoch 28/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0485
Epoch 28: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0503
Epoch 29/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0493
Epoch 29: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0516
Epoch 30/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0519
Epoch 30: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0547
Epoch 31/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0440
Epoch 31: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0519
Epoch 32/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0523
Epoch 32: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0507
Epoch 33/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0540
Epoch 33: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0523
Epoch 34/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0459
Epoch 34: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0499
Epoch 35/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0462
Epoch 35: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0510
Epoch 36/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0613
Epoch 36: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0535
Epoch 37/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0495
Epoch 37: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0518
Epoch 38/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0557
Epoch 38: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0506
Epoch 39/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0565
Epoch 39: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0529
Epoch 40/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0457
Epoch 40: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0504
Epoch 41/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0482
Epoch 41: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0504
```

```
Epoch 42/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0640
Epoch 42: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0527
Epoch 43/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0562
Epoch 43: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0547
Epoch 44/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0683
Epoch 44: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0565
Epoch 45/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0475
Epoch 45: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0508
Epoch 46/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0508
Epoch 46: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0510
Epoch 47/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0551
Epoch 47: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0523
Epoch 48/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0528
Epoch 48: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0534
Epoch 49/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0532
Epoch 49: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0517
Epoch 50/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0456
Epoch 50: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0498
Epoch 51/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0522
Epoch 51: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0509
Epoch 52/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0576
Epoch 52: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0520
Epoch 53/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0535
Epoch 53: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0535
Epoch 54/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0518
Epoch 54: loss improved from 0.04936 to 0.04936, saving model to best_model.
keras
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0494
Epoch 55/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0459
Epoch 55: loss did not improve from 0.04936
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0505
Epoch 56/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0453
Epoch 56: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0512
Epoch 57/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0486
Epoch 57: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0510
Epoch 58/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0374
Epoch 58: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0507
Epoch 59/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0449
Epoch 59: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0512
Epoch 60/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0501
Epoch 60: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0496
Epoch 61/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0525
Epoch 61: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0514
Epoch 62/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0408
Epoch 62: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0506
Epoch 63/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0530
Epoch 63: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0515
Epoch 64/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0538
Epoch 64: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0514
Epoch 65/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0458
Epoch 65: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0496
Epoch 66/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0492
Epoch 66: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0507
Epoch 67/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0537
Epoch 67: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0522
Epoch 68/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0571
Epoch 68: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0509
Epoch 69/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0536
Epoch 69: loss did not improve from 0.04936
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0504
Epoch 70/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0482
Epoch 70: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0521
Epoch 71/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0493
Epoch 71: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0517
Epoch 72/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0542
Epoch 72: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0509
Epoch 73/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0530
Epoch 73: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0498
Epoch 74/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0447
Epoch 74: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0514
Epoch 75/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0488
Epoch 75: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0501
Epoch 76/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0541
Epoch 76: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0516
Epoch 77/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0513
Epoch 77: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0498
Epoch 78/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0537
Epoch 78: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0501
Epoch 79/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0524
Epoch 79: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0529
Epoch 80/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0418
Epoch 80: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0496
Epoch 81/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0512
Epoch 81: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.0509
Epoch 82/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0489
Epoch 82: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0501
Epoch 83/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0627
Epoch 83: loss did not improve from 0.04936
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0532
Epoch 84/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0491
Epoch 84: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0525
Epoch 85/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0492
Epoch 85: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0496
Epoch 86/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0593
Epoch 86: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0513
Epoch 87/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0558
Epoch 87: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0500
Epoch 88/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0553
Epoch 88: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0516
Epoch 89/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0476
Epoch 89: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0515
Epoch 90/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0495
Epoch 90: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0503
Epoch 91/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0563
Epoch 91: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0497
Epoch 92/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0581
Epoch 92: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0510
Epoch 93/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0486
Epoch 93: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0531
Epoch 94/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0532
Epoch 94: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0514
Epoch 95/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0411
Epoch 95: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0499
Epoch 96/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0520
Epoch 96: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0512
Epoch 97/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0464
Epoch 97: loss did not improve from 0.04936
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0497
Epoch 98/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0533
Epoch 98: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0522
Epoch 99/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0556
Epoch 99: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0508
Epoch 100/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0398
Epoch 100: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0521
Epoch 101/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0587
Epoch 101: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0506
Epoch 102/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0504
Epoch 102: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0508
Epoch 103/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0489
Epoch 103: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0501
Epoch 104/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0565
Epoch 104: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0516
Epoch 105/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0548
Epoch 105: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0513
Epoch 106/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0457
Epoch 106: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0517
Epoch 107/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0464
Epoch 107: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0506
Epoch 108/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0491
Epoch 108: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0541
Epoch 109/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0615
Epoch 109: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0517
Epoch 110/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0473
Epoch 110: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0506
Epoch 111/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0516
Epoch 111: loss did not improve from 0.04936
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0497
Epoch 112/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0487
Epoch 112: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0513
Epoch 113/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0571
Epoch 113: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0503
Epoch 114/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0543
Epoch 114: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0525
Epoch 115/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0520
Epoch 115: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0500
Epoch 116/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0547
Epoch 116: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0499
Epoch 117/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0497
Epoch 117: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0517
Epoch 118/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0476
Epoch 118: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0507
Epoch 119/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0452
Epoch 119: loss did not improve from 0.04936
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0497
Epoch 120/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0487
Epoch 120: loss improved from 0.04936 to 0.04933, saving model to best_mode
l.keras
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 104ms/step - loss: 0.0493
Epoch 121/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - loss: 0.0514
Epoch 121: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0513
Epoch 122/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0513
Epoch 122: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0522
Epoch 123/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0554
Epoch 123: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0508
Epoch 124/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0544
Epoch 124: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0511
Epoch 125/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0520
```

```
Epoch 125: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0524
Epoch 126/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0575
Epoch 126: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0515
Epoch 127/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0552
Epoch 127: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0524
Epoch 128/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0589
Epoch 128: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0504
Epoch 129/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0497
Epoch 129: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.0511
Epoch 130/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0498
Epoch 130: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0499
Epoch 131/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0548
Epoch 131: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0515
Epoch 132/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0604
Epoch 132: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0531
Epoch 133/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0512
Epoch 133: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0504
Epoch 134/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0456
Epoch 134: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0501
Epoch 135/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0521
Epoch 135: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0513
Epoch 136/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0517
Epoch 136: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0498
Epoch 137/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0601
Epoch 137: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0524
Epoch 138/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0520
Epoch 138: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0507
Epoch 139/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0562
```

```
Epoch 139: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0521
Epoch 140/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0503
Epoch 140: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0505
Epoch 141/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0418
Epoch 141: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0511
Epoch 142/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0592
Epoch 142: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0544
Epoch 143/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0465
Epoch 143: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0509
Epoch 144/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0535
Epoch 144: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0512
Epoch 145/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0476
Epoch 145: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0497
Epoch 146/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0478
Epoch 146: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0507
Epoch 147/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0529
Epoch 147: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0498
Epoch 148/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0620
Epoch 148: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0500
Epoch 149/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0466
Epoch 149: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0523
Epoch 150/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0522
Epoch 150: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0498
Epoch 151/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0587
Epoch 151: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0508
Epoch 152/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0615
Epoch 152: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0529
Epoch 153/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0402
```

```
Epoch 153: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0498
Epoch 154/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0550
Epoch 154: loss did not improve from 0.04933
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0528
Epoch 155/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0396
Epoch 155: loss improved from 0.04933 to 0.04925, saving model to best_mode
l.keras
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0493
Epoch 156/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0502
Epoch 156: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0504
Epoch 157/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0526
Epoch 157: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0511
Epoch 158/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0579
Epoch 158: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0527
Epoch 159/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0654
Epoch 159: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0558
Epoch 160/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0721
Epoch 160: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0557
Epoch 161/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0596
Epoch 161: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0525
Epoch 162/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0423
Epoch 162: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0511
Epoch 163/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0542
Epoch 163: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0510
Epoch 164/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0564
Epoch 164: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0543
Epoch 165/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0568
Epoch 165: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0536
Epoch 166/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0475
Epoch 166: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0519
Epoch 167/600
```

```
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0557
Epoch 167: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0518
Epoch 168/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0527
Epoch 168: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0540
Epoch 169/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0496
Epoch 169: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0499
Epoch 170/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0443
Epoch 170: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0500
Epoch 171/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0513
Epoch 171: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0510
Epoch 172/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0573
Epoch 172: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0506
Epoch 173/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0475
Epoch 173: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0504
Epoch 174/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0409
Epoch 174: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0501
Epoch 175/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0566
Epoch 175: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0525
Epoch 176/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0520
Epoch 176: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0516
Epoch 177/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0517
Epoch 177: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0520
Epoch 178/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0511
Epoch 178: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0529
Epoch 179/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0457
Epoch 179: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0511
Epoch 180/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0580
Epoch 180: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0537
Epoch 181/600
```

```
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0498
Epoch 181: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0504
Epoch 182/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0475
Epoch 182: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0511
Epoch 183/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0555
Epoch 183: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0502
Epoch 184/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0575
Epoch 184: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0502
Epoch 185/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0531
Epoch 185: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0506
Epoch 186/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0514
Epoch 186: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0518
Epoch 187/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0481
Epoch 187: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0508
Epoch 188/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0441
Epoch 188: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0505
Epoch 189/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0503
Epoch 189: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0522
Epoch 190/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0525
Epoch 190: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0532
Epoch 191/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0478
Epoch 191: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0512
Epoch 192/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0524
Epoch 192: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0523
Epoch 193/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0421
Epoch 193: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0501
Epoch 194/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0511
Epoch 194: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0500
Epoch 195/600
```

```
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0484
Epoch 195: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0511
Epoch 196/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0413
Epoch 196: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0495
Epoch 197/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0504
Epoch 197: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0504
Epoch 198/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0575
Epoch 198: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0532
Epoch 199/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0535
Epoch 199: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0501
Epoch 200/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0521
Epoch 200: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0500
Epoch 201/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0505
Epoch 201: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0512
Epoch 202/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0539
Epoch 202: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0516
Epoch 203/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0460
Epoch 203: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0509
Epoch 204/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0397
Epoch 204: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0498
Epoch 205/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0487
Epoch 205: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0534
Epoch 206/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0560
Epoch 206: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0549
Epoch 207/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0465
Epoch 207: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0506
Epoch 208/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0504
Epoch 208: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0512
Epoch 209/600
```

```
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0485
Epoch 209: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0530
Epoch 210/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0434
Epoch 210: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0497
Epoch 211/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0462
Epoch 211: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0503
Epoch 212/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0495
Epoch 212: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0513
Epoch 213/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0648
Epoch 213: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0524
Epoch 214/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0453
Epoch 214: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0510
Epoch 215/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0576
Epoch 215: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0505
Epoch 216/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0476
Epoch 216: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0505
Epoch 217/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0552
Epoch 217: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0517
Epoch 218/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0499
Epoch 218: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0515
Epoch 219/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0551
Epoch 219: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0503
Epoch 220/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0595
Epoch 220: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0521
Epoch 221/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0619
Epoch 221: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0538
Epoch 222/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0497
Epoch 222: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0542
Epoch 223/600
```

```
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0557
Epoch 223: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0515
Epoch 224/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0572
Epoch 224: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0514
Epoch 225/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0477
Epoch 225: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0497
Epoch 226/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0477
Epoch 226: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0506
Epoch 227/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0500
Epoch 227: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0523
Epoch 228/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0543
Epoch 228: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0539
Epoch 229/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0536
Epoch 229: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0555
Epoch 230/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0601
Epoch 230: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.0559
Epoch 231/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0449
Epoch 231: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0514
Epoch 232/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0535
Epoch 232: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0501
Epoch 233/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0508
Epoch 233: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0503
Epoch 234/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0507
Epoch 234: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0519
Epoch 235/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0552
Epoch 235: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0544
Epoch 236/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0577
Epoch 236: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0534
Epoch 237/600
```

```
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0441
Epoch 237: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0498
Epoch 238/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0536
Epoch 238: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0514
Epoch 239/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0487
Epoch 239: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0530
Epoch 240/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0555
Epoch 240: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0545
Epoch 241/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0553
Epoch 241: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0500
Epoch 242/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0466
Epoch 242: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0499
Epoch 243/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0479
Epoch 243: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0511
Epoch 244/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0453
Epoch 244: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0505
Epoch 245/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0540
Epoch 245: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0528
Epoch 246/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0544
Epoch 246: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0496
Epoch 247/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0458
Epoch 247: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0498
Epoch 248/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0467
Epoch 248: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0497
Epoch 249/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0643
Epoch 249: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0515
Epoch 250/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0573
Epoch 250: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0511
Epoch 251/600
```

```
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0555
Epoch 251: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0504
Epoch 252/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0446
Epoch 252: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0504
Epoch 253/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0450
Epoch 253: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0507
Epoch 254/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0456
Epoch 254: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0503
Epoch 255/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0592
Epoch 255: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0521
Epoch 256/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0620
Epoch 256: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0523
Epoch 257/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0588
Epoch 257: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0503
Epoch 258/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0559
Epoch 258: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0535
Epoch 259/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0453
Epoch 259: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0522
Epoch 260/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0595
Epoch 260: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0525
Epoch 261/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0537
Epoch 261: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0532
Epoch 262/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0465
Epoch 262: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.0494
Epoch 263/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0535
Epoch 263: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0503
Epoch 264/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0489
Epoch 264: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0499
Epoch 265/600
```

```
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0462
Epoch 265: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0500
Epoch 266/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0481
Epoch 266: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0502
Epoch 267/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0540
Epoch 267: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0498
Epoch 268/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0455
Epoch 268: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0519
Epoch 269/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0522
Epoch 269: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0520
Epoch 270/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0458
Epoch 270: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0504
Epoch 271/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0547
Epoch 271: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0519
Epoch 272/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0553
Epoch 272: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.0501
Epoch 273/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0500
Epoch 273: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0503
Epoch 274/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0530
Epoch 274: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0495
Epoch 275/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0526
Epoch 275: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0504
Epoch 276/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - loss: 0.0551
Epoch 276: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0503
Epoch 277/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0575
Epoch 277: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0514
Epoch 278/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0517
Epoch 278: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0504
Epoch 279/600
```

```
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0459
Epoch 279: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0499
Epoch 280/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0508
Epoch 280: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0511
Epoch 281/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0536
Epoch 281: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0524
Epoch 282/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0549
Epoch 282: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0509
Epoch 283/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0577
Epoch 283: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0510
Epoch 284/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0525
Epoch 284: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0499
Epoch 285/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0513
Epoch 285: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0515
Epoch 286/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0559
Epoch 286: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0502
Epoch 287/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0472
Epoch 287: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0515
Epoch 288/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0502
Epoch 288: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0499
Epoch 289/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0518
Epoch 289: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0504
Epoch 290/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0482
Epoch 290: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0514
Epoch 291/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0447
Epoch 291: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0503
Epoch 292/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0607
Epoch 292: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0527
Epoch 293/600
```

```
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0523
Epoch 293: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0519
Epoch 294/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0530
Epoch 294: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0544
Epoch 295/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0625
Epoch 295: loss did not improve from 0.04925
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0561
Epoch 296/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0497
Epoch 296: loss improved from 0.04925 to 0.04914, saving model to best_mode
l.keras
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0491
Epoch 297/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0566
Epoch 297: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0515
Epoch 298/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0485
Epoch 298: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0505
Epoch 299/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0629
Epoch 299: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0524
Epoch 300/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0499
Epoch 300: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0493
Epoch 301/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0540
Epoch 301: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0502
Epoch 302/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0490
Epoch 302: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0501
Epoch 303/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0486
Epoch 303: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0503
Epoch 304/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0479
Epoch 304: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0503
Epoch 305/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0494
Epoch 305: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0528
Epoch 306/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0490
Epoch 306: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0506
```

```
Epoch 307/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0516
Epoch 307: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0496
Epoch 308/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0533
Epoch 308: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0512
Epoch 309/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0557
Epoch 309: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0522
Epoch 310/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0474
Epoch 310: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0503
Epoch 311/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0562
Epoch 311: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0528
Epoch 312/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0570
Epoch 312: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0494
Epoch 313/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0572
Epoch 313: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0527
Epoch 314/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0542
Epoch 314: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0513
Epoch 315/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0402
Epoch 315: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0499
Epoch 316/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0530
Epoch 316: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0506
Epoch 317/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0491
Epoch 317: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0498
Epoch 318/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0500
Epoch 318: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0509
Epoch 319/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0527
Epoch 319: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0506
Epoch 320/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0596
Epoch 320: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0501
```

```
Epoch 321/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0420
Epoch 321: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0510
Epoch 322/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0583
Epoch 322: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0539
Epoch 323/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0480
Epoch 323: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0507
Epoch 324/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0467
Epoch 324: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0514
Epoch 325/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0370
Epoch 325: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0511
Epoch 326/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0422
Epoch 326: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0495
Epoch 327/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0557
Epoch 327: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0496
Epoch 328/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0465
Epoch 328: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0496
Epoch 329/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0443
Epoch 329: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0512
Epoch 330/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0446
Epoch 330: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0502
Epoch 331/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0559
Epoch 331: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0518
Epoch 332/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0475
Epoch 332: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0503
Epoch 333/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0442
Epoch 333: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0505
Epoch 334/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0548
Epoch 334: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0517
```

```
Epoch 335/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0453
Epoch 335: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0535
Epoch 336/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0451
Epoch 336: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0518
Epoch 337/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0500
Epoch 337: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0525
Epoch 338/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0502
Epoch 338: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0508
Epoch 339/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0516
Epoch 339: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0506
Epoch 340/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0587
Epoch 340: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0518
Epoch 341/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0524
Epoch 341: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0523
Epoch 342/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0514
Epoch 342: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0503
Epoch 343/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0517
Epoch 343: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0508
Epoch 344/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0550
Epoch 344: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0502
Epoch 345/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0567
Epoch 345: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0503
Epoch 346/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0601
Epoch 346: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0502
Epoch 347/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0521
Epoch 347: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0513
Epoch 348/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0541
Epoch 348: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0527
```

```
Epoch 349/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0569
Epoch 349: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0501
Epoch 350/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0438
Epoch 350: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0507
Epoch 351/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0541
Epoch 351: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0537
Epoch 352/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0533
Epoch 352: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.0503
Epoch 353/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0573
Epoch 353: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0501
Epoch 354/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0535
Epoch 354: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0506
Epoch 355/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0594
Epoch 355: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0525
Epoch 356/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0555
Epoch 356: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0518
Epoch 357/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0528
Epoch 357: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0503
Epoch 358/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0480
Epoch 358: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0513
Epoch 359/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0580
Epoch 359: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0538
Epoch 360/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0549
Epoch 360: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0534
Epoch 361/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0468
Epoch 361: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0504
Epoch 362/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0569
Epoch 362: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0511
```

```
Epoch 363/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0461
Epoch 363: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0504
Epoch 364/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0483
Epoch 364: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0503
Epoch 365/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0528
Epoch 365: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0506
Epoch 366/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0486
Epoch 366: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0538
Epoch 367/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0490
Epoch 367: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0536
Epoch 368/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0514
Epoch 368: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0529
Epoch 369/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0355
Epoch 369: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0506
Epoch 370/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0418
Epoch 370: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0516
Epoch 371/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0637
Epoch 371: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0523
Epoch 372/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0538
Epoch 372: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0540
Epoch 373/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0580
Epoch 373: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0515
Epoch 374/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0555
Epoch 374: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0495
Epoch 375/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0408
Epoch 375: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0515
Epoch 376/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0467
Epoch 376: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0509
```

```
Epoch 377/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0466
Epoch 377: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0514
Epoch 378/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0565
Epoch 378: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0533
Epoch 379/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0586
Epoch 379: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0553
Epoch 380/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0546
Epoch 380: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0535
Epoch 381/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0567
Epoch 381: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0541
Epoch 382/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0503
Epoch 382: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0501
Epoch 383/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0483
Epoch 383: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0515
Epoch 384/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0560
Epoch 384: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0496
Epoch 385/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0566
Epoch 385: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0510
Epoch 386/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0534
Epoch 386: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0509
Epoch 387/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0458
Epoch 387: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0497
Epoch 388/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0583
Epoch 388: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0509
Epoch 389/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0577
Epoch 389: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0495
Epoch 390/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0558
Epoch 390: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0517
```

```
Epoch 391/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0535
Epoch 391: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0509
Epoch 392/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0538
Epoch 392: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0508
Epoch 393/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0598
Epoch 393: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0530
Epoch 394/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0511
Epoch 394: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0513
Epoch 395/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0544
Epoch 395: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0511
Epoch 396/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0590
Epoch 396: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0521
Epoch 397/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0530
Epoch 397: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0501
Epoch 398/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0513
Epoch 398: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0508
Epoch 399/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0528
Epoch 399: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0509
Epoch 400/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0532
Epoch 400: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0526
Epoch 401/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0440
Epoch 401: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0498
Epoch 402/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0540
Epoch 402: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0506
Epoch 403/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0551
Epoch 403: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0519
Epoch 404/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0519
Epoch 404: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0495
```

```
Epoch 405/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0530
Epoch 405: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0503
Epoch 406/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0434
Epoch 406: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0501
Epoch 407/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0456
Epoch 407: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0513
Epoch 408/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0625
Epoch 408: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0528
Epoch 409/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0565
Epoch 409: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0535
Epoch 410/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0522
Epoch 410: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0499
Epoch 411/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0564
Epoch 411: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0501
Epoch 412/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0545
Epoch 412: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0512
Epoch 413/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0460
Epoch 413: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0499
Epoch 414/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0464
Epoch 414: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0506
Epoch 415/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0520
Epoch 415: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0494
Epoch 416/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0601
Epoch 416: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0506
Epoch 417/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0557
Epoch 417: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0525
Epoch 418/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0561
Epoch 418: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0533
```

```
Epoch 419/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0557
Epoch 419: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0500
Epoch 420/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0445
Epoch 420: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0505
Epoch 421/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0486
Epoch 421: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0508
Epoch 422/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0528
Epoch 422: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0511
Epoch 423/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0509
Epoch 423: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0504
Epoch 424/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0478
Epoch 424: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0501
Epoch 425/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0521
Epoch 425: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0514
Epoch 426/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0667
Epoch 426: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0539
Epoch 427/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0502
Epoch 427: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0496
Epoch 428/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0483
Epoch 428: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0505
Epoch 429/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0565
Epoch 429: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0504
Epoch 430/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0553
Epoch 430: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0504
Epoch 431/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0561
Epoch 431: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0522
Epoch 432/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0540
Epoch 432: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0515
```

```
Epoch 433/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0563
Epoch 433: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0530
Epoch 434/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0533
Epoch 434: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0494
Epoch 435/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0517
Epoch 435: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0523
Epoch 436/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0515
Epoch 436: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0502
Epoch 437/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0594
Epoch 437: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0528
Epoch 438/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0469
Epoch 438: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0539
Epoch 439/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0550
Epoch 439: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0504
Epoch 440/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0396
Epoch 440: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0503
Epoch 441/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0542
Epoch 441: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0501
Epoch 442/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0461
Epoch 442: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0495
Epoch 443/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0484
Epoch 443: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0499
Epoch 444/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0434
Epoch 444: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0510
Epoch 445/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0569
Epoch 445: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0535
Epoch 446/600
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0626
Epoch 446: loss did not improve from 0.04914
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0547
```

```
Epoch 446: early stopping
Restoring model weights from the end of the best epoch: 296.
best epoch =  296
smallest loss = 0.04914478957653046
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step
```

Cell 5

In [6]:

```
model = keras.models.load_model('best_model.keras')
test = []
outpt=[]
predictions = model.predict(data_inputs)

#first point (row [0])comparison of data and prediction
# put in a loop to print comparion for all data points

test = [[ xarray[0][0] , xarray[0][1] , xarray[0][2] ]]
testarray = np.array(test)
outpt = model.predict(testarray)
print ('row [0] data:  T1= ', xarray[0][0]*Tmed, ', gam= ', xarray[0][1]*gam
    ', qsol= ', xarray[0][2]*qsmed,', alpha= ', yarray[0][0]*almed,\
    ',  predicted alpha = ', outpt[0][0]*almed)

#SETTING UP PLOT
%matplotlib inline
# importing the required module
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [8, 8] # for square canvas
#========
'''CALCULATE PREDICTED VALUES AND RETRIEVE DATA VALUES TO PLOT'''
plt.scatter(predictions[:, 0], data_outputs[:, 0])
plt.title('Alpha: Actual vs Predicted Values')
plt.xlabel('Predicted α (normalized)')
plt.ylabel('Actual α (normalized)')
plt.loglog()
plt.xlim(xmax = 10, xmin = 0.1)
plt.ylim(ymax = 10, ymin = 0.1)
# Generate red y=x line
x_data = np.linspace(0.1, 10.0, num=3)
y_data = x_data
plt.plot(x_data, y_data, color='red')
plt.show()

# ===== ADD THIS SURFACE PLOT CODE BELOW =====
from mpl_toolkits.mplot3d import Axes3D

# Set gamma fixed at 0.25
gam_fixed = 0.25

# Create ranges for T1 and Q_s
T1_range = np.linspace(268, 318, 20)  # 268 to 318 K
Qs_range = np.linspace(500, 2500, 20)  # 500 to 2500 kW

# Create meshgrid for surface plot
T1_mesh, Qs_mesh = np.meshgrid(T1_range, Qs_range)
```

```
# Normalize the inputs using your medians
T1_norm = T1_mesh / new_Tmed
gam_norm = np.full_like(T1_mesh, gam_fixed / new_gamed)  # fixed at 0.25
Qs_norm = Qs_mesh / new_qsmed

# Create prediction array
alpha_predictions = np.zeros_like(T1_mesh)
eta_predictions = np.zeros_like(T1_mesh)

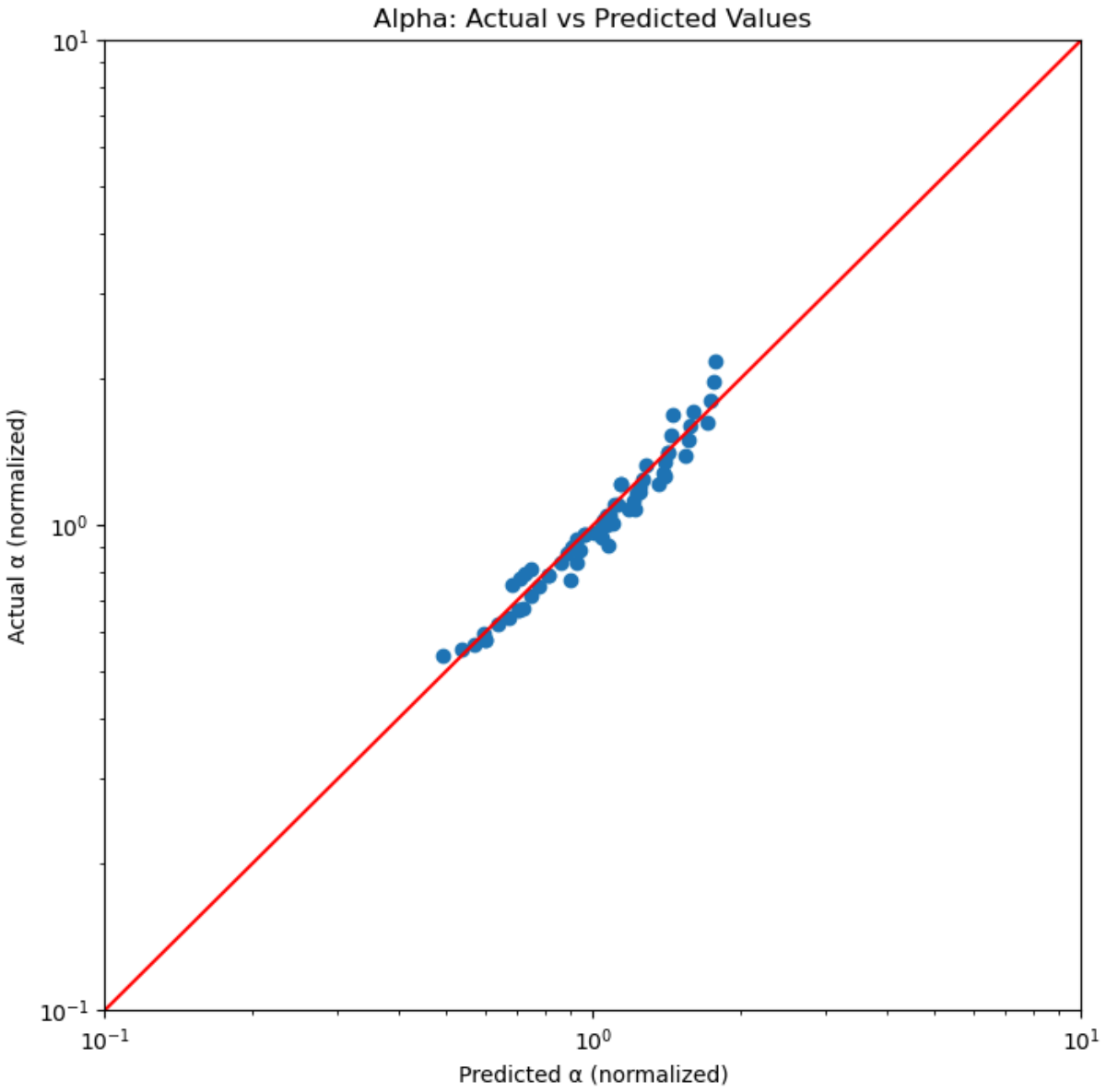
# Loop through and make predictions
for i in range(len(T1_range)):
    for j in range(len(Qs_range)):
        # Create input array [T1_norm, gam_norm, Qs_norm]
        test_input = np.array([[T1_norm[i, j], gam_norm[i, j], Qs_norm[i, j]
        # Make prediction
        prediction = model.predict(test_input, verbose=0)
        # Denormalize outputs
        alpha_predictions[i, j] = prediction[0][0] * new_almed
        eta_predictions[i, j] = prediction[0][1] * new_efmed

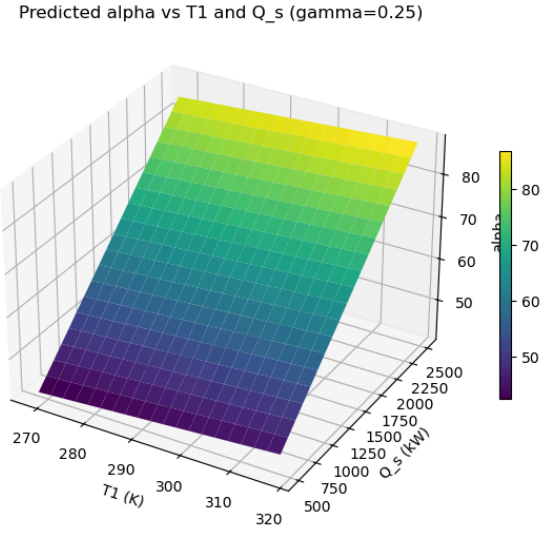
# Create surface plot for alpha
fig = plt.figure(figsize=(12, 5))

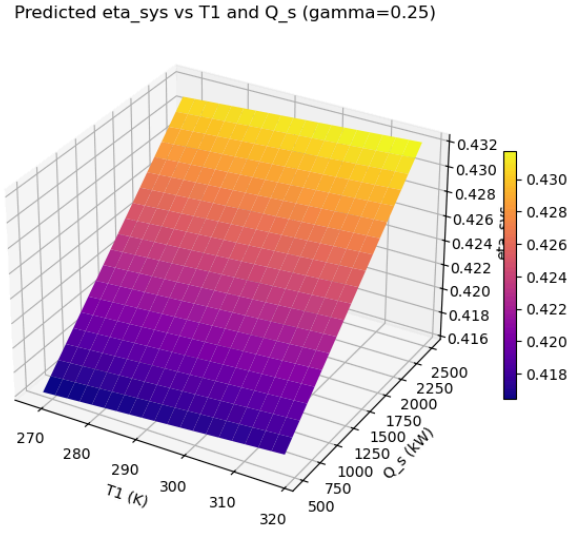
# Plot 1: alpha vs T1 and Q_s
ax1 = fig.add_subplot(121, projection='3d')
surf1 = ax1.plot_surface(T1_mesh, Qs_mesh, alpha_predictions, cmap='viridis'
ax1.set_xlabel('T1 (K)')
ax1.set_ylabel('Q_s (kW)')
ax1.set_zlabel('alpha')
ax1.set_title('Predicted alpha vs T1 and Q_s (gamma=0.25)')
fig.colorbar(surf1, ax=ax1, shrink=0.5)

# Plot 2: eta_sys vs T1 and Q_s
ax2 = fig.add_subplot(122, projection='3d')
surf2 = ax2.plot_surface(T1_mesh, Qs_mesh, eta_predictions, cmap='plasma')
ax2.set_xlabel('T1 (K)')
ax2.set_ylabel('Q_s (kW)')
ax2.set_zlabel('eta_sys')
ax2.set_title('Predicted eta_sys vs T1 and Q_s (gamma=0.25)')
fig.colorbar(surf2, ax=ax2, shrink=0.5)
plt.tight_layout()
plt.show()
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
row [0] data:  T1=  96354.0 , gam=  0.0 , qsol=  750000.0 , alpha=  2120.692
6853 ,  predicted alpha =  10824.377747146606
```


Alpha: Actual vs Predicted Values
Actual α (normalized)
Predicted α (normalized)
10^{1}
10^{0}
10^{-1}


Predicted alpha vs T1 and Q_s (gamma=0.25)
T1 (K)
Q_s (kW)
alpha


Predicted eta_sys vs T1 and Q_s (gamma=0.25)
T1 (K)
Q_s (kW)

Task 2.3

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
from sklearn.metrics import mean_squared_error

# ============================================
# TASK 2.3 - Test Data Prediction and Analysis
# ============================================

# Load the trained model (make sure 'best_model.keras' is in your directory)
model = keras.models.load_model('best_model.keras')

# Test data from the PDF (page 5)
test_inputs_raw = np.array([
    [318.0, 0.0, 500.0],
    [318.0, 0.0, 1500.0],
    [318.0, 0.0, 2500.0],
    [318.0, 0.25, 1500.0],
    [318.0, 0.5, 500.0],
    [318.0, 0.5, 1500.0],
    [318.0, 0.5, 2500.0],
    [303.0, 0.0, 1000.0],
    [303.0, 0.0, 2000.0],
    [303.0, 0.25, 1000.0],
    [303.0, 0.25, 2000.0],
    [303.0, 0.5, 1000.0],
    [303.0, 0.5, 2000.0],
    [288.0, 0.0, 500.0],
    [288.0, 0.0, 2500.0],
    [288.0, 0.25, 2500.0],
    [288.0, 0.5, 1500.0],
    [269.0, 0.0, 1500.0],
    [268.0, 0.25, 2000.0],
    [268.0, 0.5, 2500.0]
])

# Actual test outputs [alpha, eta_sys]
test_outputs_actual = np.array([
    [35.13, 0.3808],
    [47.46, 0.3930],
    [73.12, 0.4061],
    [66.34, 0.4098],
    [63.09, 0.4154],
    [85.23, 0.4197],
    [131.32, 0.4242],
    [38.99, 0.4012],
    [53.80, 0.4136],
    [54.51, 0.4215],
    [75.22, 0.4290],
    [70.04, 0.4337],
    [96.65, 0.4382],
    [33.45, 0.4091],
    [60.80, 0.4334],
    [85.044, 0.4477],
    [77.56, 0.4516],
```

```
    [40.68, 0.4393],
    [65.24, 0.4628],
    [98.23, 0.4760]
])

# ============================================
# IMPORTANT: UPDATE THESE MEDIAN VALUES
# ============================================
# These MUST be the exact same median values used during training
# Check your cells 1-4 in your notebook for the correct values!

# Based on typical values from similar problems (YOU MUST UPDATE THESE):
Tmed = new_Tmed
gamed = new_gamed
qsmed = new_qsmed
almed = new_almed
efmed = new_efmed

print("=" * 60)
print("MEDIAN VALUES BEING USED (VERIFY THESE ARE CORRECT!):")
print("=" * 60)
print(f"Tmed = {Tmed} K")
print(f"gamed = {gamed}")
print(f"qsmed = {qsmed} kW")
print(f"almed = {almed}")
print(f"efmed = {efmed}")
print("=" * 60)
print()

# ============================================
# Step 1: Normalize the test inputs
# ============================================
print("Normalizing test inputs...")
test_inputs_normalized = np.zeros_like(test_inputs_raw)
test_inputs_normalized[:, 0] = test_inputs_raw[:, 0] / Tmed
test_inputs_normalized[:, 1] = test_inputs_raw[:, 1] / gamed
test_inputs_normalized[:, 2] = test_inputs_raw[:, 2] / qsmed

print(f"Sample normalized input (first row): {test_inputs_normalized[0]}")
print()

# ============================================
# Step 2: Make predictions
# ============================================
print("Making predictions on test data...")
predictions_normalized = model.predict(test_inputs_normalized)

# ============================================
# Step 3: Denormalize the predictions
# ============================================
predictions_actual = np.zeros_like(predictions_normalized)
predictions_actual[:, 0] = predictions_normalized[:, 0] * almed  # Denormali
predictions_actual[:, 1] = predictions_normalized[:, 1] * efmed  # Denormali

print(f"Sample prediction (first row): α={predictions_actual[0, 0]:.2f}, η_s
print(f"Sample actual (first row): α={test_outputs_actual[0, 0]:.2f}, η_sys=
```

```
print()

# ============================================
# Step 4: Create log-log plot for alpha
# ============================================
plt.figure(figsize=(8, 8))

# Extract alpha values
actual_alpha = test_outputs_actual[:, 0]
predicted_alpha = predictions_actual[:, 0]

# Create scatter plot
plt.scatter(actual_alpha, predicted_alpha, s=60, alpha=0.7,
           edgecolors='black', linewidth=1, color='blue', label='Test Data')

# Add perfect prediction line (y=x)
min_val = min(actual_alpha.min(), predicted_alpha.min()) * 0.9
max_val = max(actual_alpha.max(), predicted_alpha.max()) * 1.1
perfect_line = np.logspace(np.log10(min_val), np.log10(max_val), 100)
plt.plot(perfect_line, perfect_line, 'r-', linewidth=2, label='Perfect Predi

# Set log scale
plt.xscale('log')
plt.yscale('log')

# Labels and formatting
plt.xlabel('Actual α (from test data)', fontsize=12)
plt.ylabel('Predicted α (from neural network)', fontsize=12)
plt.title('Alpha: Actual vs Predicted (Log-Log Scale)', fontsize=14, fontwei
plt.grid(True, which="both", ls="-", alpha=0.2)
plt.legend(fontsize=10)

# Set limits to match the instructions
plt.xlim(10, 200)
plt.ylim(10, 200)

plt.tight_layout()
plt.savefig('alpha_loglog_plot.png', dpi=150, bbox_inches='tight')
plt.show()

# ============================================
# Step 5: Calculate RMS Deviation
# ============================================
print("=" * 60)
print("RMS DEVIATION ANALYSIS FOR ALPHA")
print("=" * 60)

# Calculate RMS for alpha
rms_alpha = np.sqrt(mean_squared_error(actual_alpha, predicted_alpha))
print(f"RMS deviation for alpha: {rms_alpha:.4f}")

# Calculate relative RMS error (percentage)
relative_rms_alpha = (rms_alpha / np.mean(actual_alpha)) * 100
print(f"Relative RMS error for alpha: {relative_rms_alpha:.2f}%")

# Calculate mean absolute error
```

```
mae_alpha = np.mean(np.abs(actual_alpha - predicted_alpha))
print(f"Mean Absolute Error for alpha: {mae_alpha:.4f}")

# Calculate R² score
from sklearn.metrics import r2_score
r2_alpha = r2_score(actual_alpha, predicted_alpha)
print(f"R² score for alpha: {r2_alpha:.4f}")

print("=" * 60)
print()

# ============================================
# Step 6: Detailed comparison table
# ============================================
print("=" * 100)
print("DETAILED TEST DATA COMPARISON")
print("=" * 100)
print(f"{'Test #':<8} {'Input (T1, γ, Qs)':<25} {'Actual α':<12} {'Predicted
print("-" * 100)

for i in range(len(test_inputs_raw)):
    input_str = f"({test_inputs_raw[i, 0]:.0f}, {test_inputs_raw[i, 1]:.2f},
    actual = actual_alpha[i]
    predicted = predicted_alpha[i]
    error = abs(actual - predicted)
    percent_error = (error / actual) * 100

    print(f"{i+1:<8} {input_str:<25} {actual:<12.2f} {predicted:<15.2f} {err

print("=" * 100)

# ============================================
# Step 7: Additional analysis plots
# ============================================

# Create a figure with multiple subplots
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Plot 1: Residuals for alpha
ax1 = axes[0, 0]
residuals_alpha = actual_alpha - predicted_alpha
ax1.scatter(predicted_alpha, residuals_alpha, s=50, alpha=0.7, edgecolors='b
ax1.axhline(y=0, color='r', linestyle='--', alpha=0.5)
ax1.set_xlabel('Predicted α')
ax1.set_ylabel('Residuals (Actual - Predicted)')
ax1.set_title('Residual Plot for Alpha')
ax1.grid(True, alpha=0.3)

# Plot 2: Percentage Error Distribution
ax2 = axes[0, 1]
percent_errors = ((actual_alpha - predicted_alpha) / actual_alpha) * 100
ax2.hist(percent_errors, bins=10, edgecolor='black', alpha=0.7)
ax2.set_xlabel('Percentage Error (%)')
ax2.set_ylabel('Frequency')
ax2.set_title('Distribution of Percentage Errors for Alpha')
ax2.grid(True, alpha=0.3)
```

```
# Plot 3: Predicted vs Actual (linear scale)
ax3 = axes[1, 0]
ax3.scatter(actual_alpha, predicted_alpha, s=50, alpha=0.7, edgecolors='blac
ax3.plot([actual_alpha.min(), actual_alpha.max()],
         [actual_alpha.min(), actual_alpha.max()], 'r--', alpha=0.5)
ax3.set_xlabel('Actual α')
ax3.set_ylabel('Predicted α')
ax3.set_title('Alpha: Actual vs Predicted (Linear Scale)')
ax3.grid(True, alpha=0.3)

# Plot 4: Error vs Input Temperature
ax4 = axes[1, 1]
abs_errors = np.abs(actual_alpha - predicted_alpha)
ax4.scatter(test_inputs_raw[:, 0], abs_errors, s=50, alpha=0.7, edgecolors='
ax4.set_xlabel('Input Temperature T1 (K)')
ax4.set_ylabel('Absolute Error in α')
ax4.set_title('Prediction Error vs Input Temperature')
ax4.grid(True, alpha=0.3)

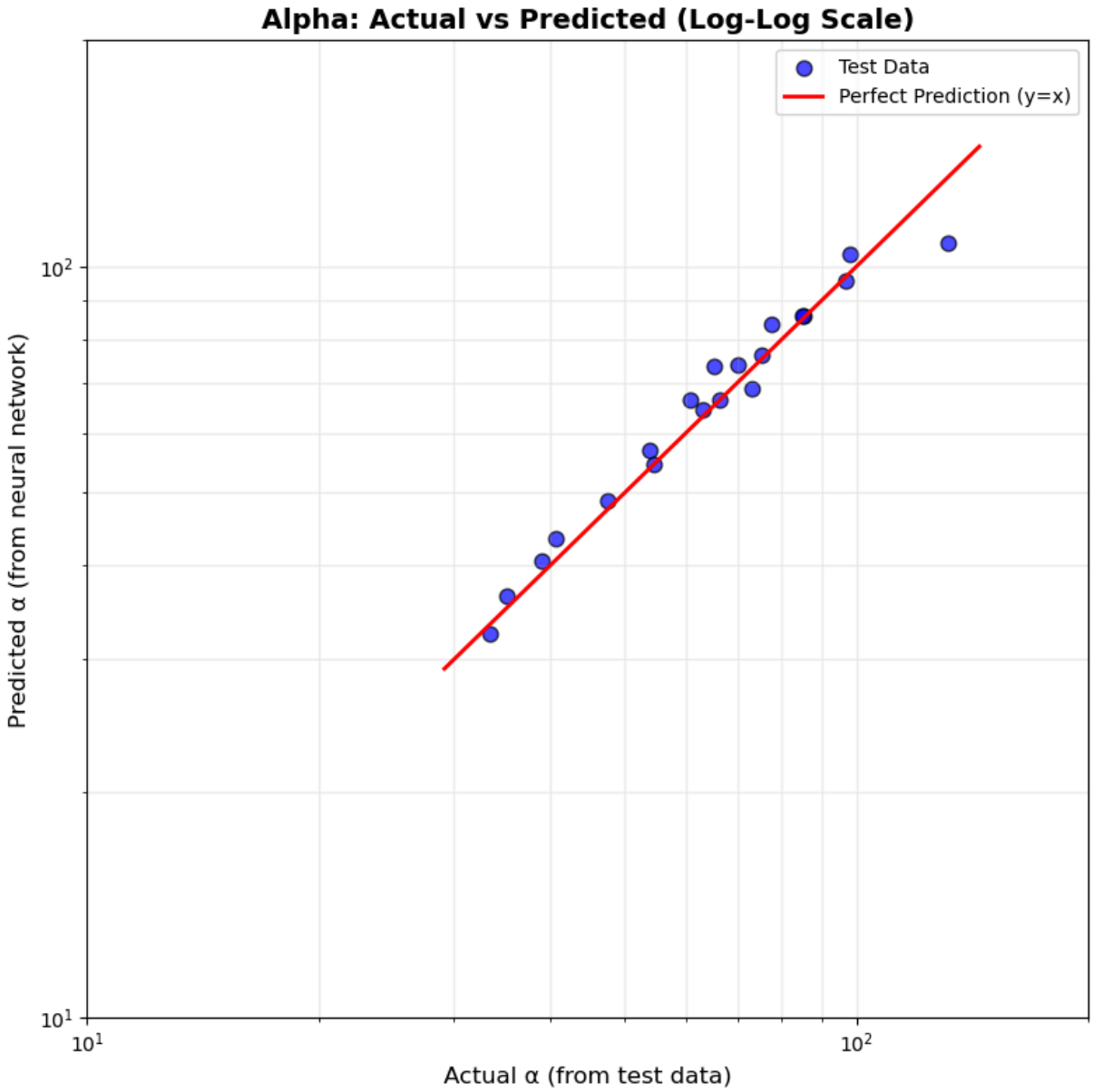
plt.tight_layout()
plt.savefig('additional_analysis_plots.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n" + "=" * 60)
print("TASK 2.3 COMPLETE!")
print("=" * 60)
print("Results saved:")
print("  - alpha_loglog_plot.png")
print("  - additional_analysis_plots.png")
print("\nIMPORTANT REMINDERS:")
print("1. Verify that the median values match those from your training!")
print("2. The log-log plot shows how well the model predicts α")
print("3. Perfect predictions would lie on the red y=x line")
print(f"4. Your RMS deviation for α is: {rms_alpha:.4f}")
print("=" * 60)
```

```
============================================================
MEDIAN VALUES BEING USED (VERIFY THESE ARE CORRECT!):
============================================================
Tmed = 303.0 K
gamed = 0.25
qsmed = 1500.0 kW
almed = 60.36425
efmed = 0.4271
============================================================

Normalizing test inputs...
Sample normalized input (first row): [1.04950495 0.         0.33333333]

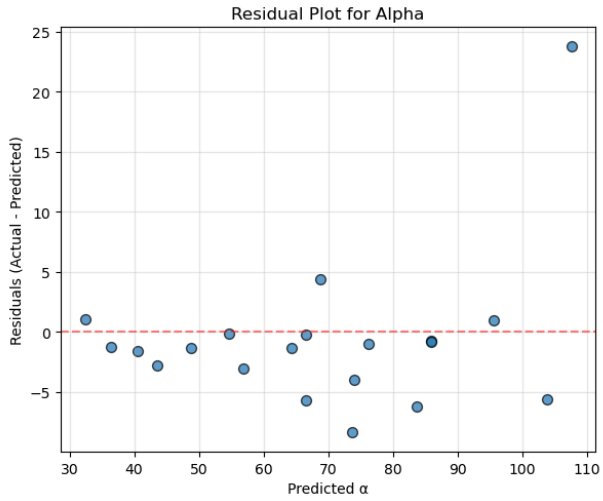
Making predictions on test data...
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 651ms/step
Sample prediction (first row): α=36.38, η_sys=0.3790
Sample actual (first row): α=35.13, η_sys=0.3808
```

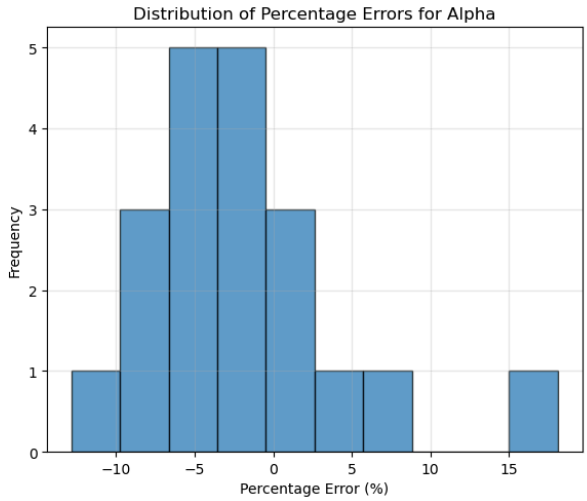

Alpha: Actual vs Predicted (Log-Log Scale)
Test Data
Perfect Prediction (y=x)
Predicted α (from neural network)
Actual α (from test data)
10^1
10^2
10^1
10^2

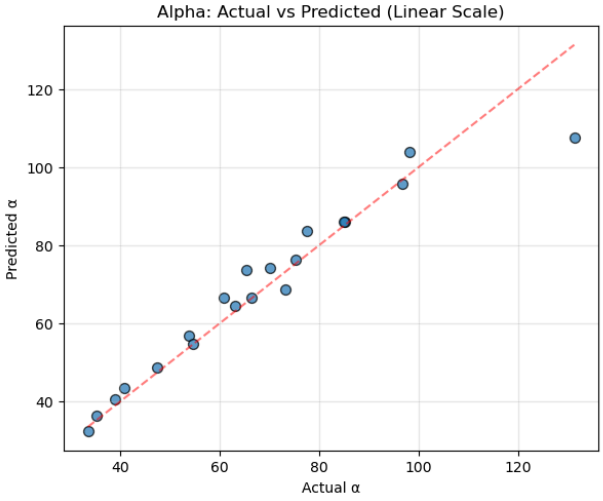
```
============================================================
RMS DEVIATION ANALYSIS FOR ALPHA
============================================================
RMS deviation for alpha: 6.3297
Relative RMS error for alpha: 9.36%
Mean Absolute Error for alpha: 3.7165
R² score for alpha: 0.9288
============================================================

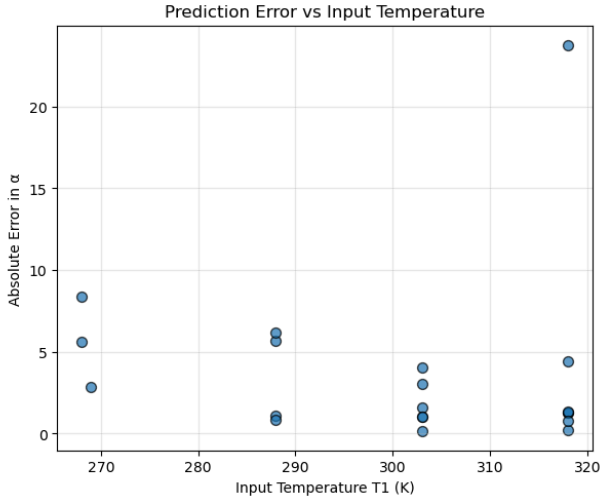
================================================================================
========================
DETAILED TEST DATA COMPARISON
================================================================================
========================
Test #   Input (T1, γ, Qs)         Actual α     Predicted α     Error
% Error
--------------------------------------------------------------------------------
------------------------
1        (318, 0.00, 500)          35.13        36.38           1.25
3.57      %
2        (318, 0.00, 1500)         47.46        48.76           1.30
2.73      %
3        (318, 0.00, 2500)         73.12        68.73           4.39
6.01      %
4        (318, 0.25, 1500)         66.34        66.56           0.22
0.33      %
5        (318, 0.50, 500)          63.09        64.40           1.31
2.07      %
6        (318, 0.50, 1500)         85.23        85.98           0.75
0.87      %
7        (318, 0.50, 2500)         131.32       107.55          23.77
18.10     %
8        (303, 0.00, 1000)         38.99        40.57           1.58
4.05      %
9        (303, 0.00, 2000)         53.80        56.82           3.02
5.61      %
10       (303, 0.25, 1000)         54.51        54.65           0.14
0.26      %
11       (303, 0.25, 2000)         75.22        76.23           1.01
1.34      %
12       (303, 0.50, 1000)         70.04        74.07           4.03
5.75      %
13       (303, 0.50, 2000)         96.65        95.64           1.01
1.04      %
14       (288, 0.00, 500)          33.45        32.38           1.07
3.19      %
15       (288, 0.00, 2500)         60.80        66.49           5.69
9.35      %
16       (288, 0.25, 2500)         85.04        85.90           0.85
1.01      %
17       (288, 0.50, 1500)         77.56        83.74           6.18
7.96      %
18       (269, 0.00, 1500)         40.68        43.49           2.81
6.91      %
19       (268, 0.25, 2000)         65.24        73.62           8.38
12.84     %
```

```
20       (268, 0.50, 2500)         98.23         103.82          5.59
5.69      %
================================================================================
========================
```










```
============================================================
TASK 2.3 COMPLETE!
============================================================
Results saved:
  - alpha_loglog_plot.png
  - additional_analysis_plots.png

IMPORTANT REMINDERS:
1. Verify that the median values match those from your training!
2. The log-log plot shows how well the model predicts α
3. Perfect predictions would lie on the red y=x line
4. Your RMS deviation for α is: 6.3297
============================================================
```

In [8]:
```
#CodeP2.5F25  ME249   V.P. Carey
#===import relevant packages
import tensorflow
import tensorflow.keras
import pygad.kerasga
import numpy as np
import pygad
import pandas as pd
#the following 2 lines are only needed for Mac OS machines
import os
os.environ['KMP_DUPLICATE_LIB_OK']='True'
#========================

#create input data array
xdata = []
xdata =  [[ 318.0 , 0.0 , 500.0 ], [ 318.0 , 0.0 , 1000.0 ]]
xdata.append([ 318.0 , 0.0 , 1500.0 ])
xdata.append([ 318.0 , 0.0 , 2000.0 ])
xdata.append([ 318.0 , 0.0 , 2500.0 ])
xdata.append([ 318.0 , 0.25 , 500.0 ])
xdata.append([ 318.0 , 0.25 , 1000.0 ])
xdata.append([ 318.0 , 0.25 , 1500.0 ])
xdata.append([ 318.0 , 0.25 , 2000.0 ])
xdata.append([ 318.0 , 0.25 , 2500.0 ])
xdata.append([ 318.0 , 0.5 , 500.0 ])
xdata.append([ 318.0 , 0.5 , 1000.0 ])
xdata.append([ 318.0 , 0.5 , 1500.0 ])
xdata.append([ 318.0 , 0.5 , 2000.0 ])
xdata.append([ 318.0 , 0.5 , 2501.0 ])

xdata.append([ 303.0 , 0.0 , 500.0 ])
xdata.append([ 303.0 , 0.0 , 1000.0 ])
xdata.append([ 303.0 , 0.0 , 1500.0 ])
xdata.append([ 303.0 , 0.0 , 2000.0 ])
xdata.append([ 303.0 , 0.0 , 2500.0 ])
xdata.append([ 303.0 , 0.25 , 500.0 ])
xdata.append([ 303.0 , 0.25 , 1000.0 ])
xdata.append([ 303.0 , 0.25 , 1500.0 ])
xdata.append([ 303.0 , 0.25 , 2000.0 ])
xdata.append([ 303.0 , 0.25 , 2500.0 ])
xdata.append([ 303.0 , 0.5 , 500.0 ])
xdata.append([ 303.0 , 0.5 , 1000.0 ])
xdata.append([ 303.0 , 0.5 , 1500.0 ])
xdata.append([ 303.0 , 0.5 , 2000.0 ])
xdata.append([ 303.0 , 0.5 , 2500.0 ])

xdata.append([ 288.0 , 0.0 , 500.0 ])
xdata.append([ 288.0 , 0.0 , 1000.0 ])
xdata.append([ 288.0 , 0.0 , 1500.0 ])
xdata.append([ 288.0 , 0.0 , 2000.0 ])
xdata.append([ 288.0 , 0.0 , 2500.0 ])
xdata.append([ 288.0 , 0.25 , 500.0 ])
xdata.append([ 288.0 , 0.25 , 1000.0 ])
xdata.append([ 288.0 , 0.25 , 1500.0 ])
```

```
xdata.append([ 288.0 , 0.25 , 2000.0 ])
xdata.append([ 288.0 , 0.25 , 2500.0 ])
xdata.append([ 288.0 , 0.51 , 500.0 ])
xdata.append([ 288.0 , 0.5 , 1000.0 ])
xdata.append([ 288.0 , 0.5 , 1500.0 ])
xdata.append([ 288.0 , 0.5 , 2000.0 ])
xdata.append([ 288.0 , 0.5 , 2500.0 ])

xdata.append([ 268.0 , 0.0 , 500.0 ])
xdata.append([ 268.0 , 0.0 , 1000.0 ])
xdata.append([ 268.0 , 0.0 , 1500.0 ])
xdata.append([ 268.0 , 0.0 , 2000.0 ])
xdata.append([ 268.0 , 0.0 , 2500.0 ])
xdata.append([ 268.0 , 0.25 , 500.0 ])
xdata.append([ 268.0 , 0.25 , 1000.0 ])
xdata.append([ 268.0 , 0.25 , 1500.0 ])
xdata.append([ 268.0 , 0.25 , 2000.0 ])
xdata.append([ 268.0 , 0.25 , 2500.0 ])
xdata.append([ 268.0 , 0.5 , 500.0 ])
xdata.append([ 268.0 , 0.5 , 1000.0 ])
xdata.append([ 268.0 , 0.5 , 1500.0 ])
xdata.append([ 268.0 , 0.5 , 2000.0 ])
xdata.append([ 268.0 , 0.5 , 2500.0 ])



#outputs
#alpha = air/fuel ratio , cycle efficiency:
ydata =  [[ 35.1316 , 0.3808 ],[ 40.3764 , 0.38686 ]]
ydata.append([ 47.4620 , 0.3930 ])
ydata.append([ 57.5639 , 0.39949 ])
ydata.append([ 73.1286 , 0.40612 ])
ydata.append([ 49.1110 , 0.4023 ])
ydata.append([ 56.4428 , 0.40605 ])
ydata.append([ 66.3479 , 0.4098 ])
ydata.append([ 80.4695 , 0.4133])
ydata.append([ 102.2276 , 0.4175 ])
ydata.append([ 63.0904 , 0.41540 ])
ydata.append([ 72.5092 , 0.4175 ])
ydata.append([ 85.2338, 0.4197 ])
ydata.append([ 103.3750 , 0.42192 ])
ydata.append([ 131.3266 , 0.4242 ])

ydata.append([ 34.273 , 0.3952 ])
ydata.append([ 38.99026 , 0.4012 ])
ydata.append([ 45.2133, 0.4073 ])
ydata.append([ 53.8000 , 0.4136 ])
ydata.append([ 66.4130 , 0.4201 ])
ydata.append([ 47.922 , 0.4178 ])
ydata.append([ 54.518 , 0.4215 ])
ydata.append([ 63.220 , 0.4252 ])
ydata.append([ 75.226 , 0.4290 ])
ydata.append([ 92.862 , 0.4329 ])
ydata.append([ 61.572 , 0.4315 ])
ydata.append([ 70.0468 , 0.43373 ])
ydata.append([ 81.226 , 0.43597 ])
```

```
ydata.append([ 96.653 , 0.4382 ])
ydata.append([ 119.3124 , 0.44045 ])

ydata.append([ 33.4521 , 0.40913 ])
ydata.append([ 37.6911, 0.4150 ])
ydata.append([ 43.1602 , 0.4209 ])
ydata.append([ 50.4858 , 0.4271 ])
ydata.append([ 60.8067 , 0.4334 ])
ydata.append([ 46.7865 , 0.4328 ])
ydata.append([ 52.7151 , 0.43646 ])
ydata.append([ 60.36425 , 0.44016 ])
ydata.append([ 70.6099 , 0.443926 ])
ydata.append([ 85.0447 , 0.4477 ])
ydata.append([ 60.1208 , 0.44721 ])
ydata.append([ 67.7391 , 0.44940 ])
ydata.append([ 77.56830 , 0.4516 ])
ydata.append([ 90.73410 , 0.4538 ])
ydata.append([ 109.2828 , 0.4560 ])

ydata.append([ 32.4123 , 0.42694 ])
ydata.append([ 36.0807 , 0.4325 ])
ydata.append([ 40.6854 , 0.4383 ])
ydata.append([ 46.6374 , 0.4442 ])
ydata.append([ 54.6293 , 0.4503 ])
ydata.append([ 45.3472 , 0.4519 ])
ydata.append([ 50.4796 , 0.4555 ])
ydata.append([ 56.9219 , 0.4591 ])
ydata.append([ 65.2492 , 0.4628 ])
ydata.append([ 76.4304 , 0.4665 ])
ydata.append([ 58.2822 , 0.4672 ])
ydata.append([ 64.8785 , 0.4693 ])
ydata.append([ 73.1584 , 0.4715 ])
ydata.append([ 83.8610 , 0.4738 ])
ydata.append([ 98.2316 , 0.4760 ])

print(xdata)
print(ydata)


xdata = np.array(xdata)
ydata = np.array(ydata)

# Calculate medians once
medians_x = np.array([np.median(xdata[:, 0]),
                      np.median(xdata[:, 1]),
                      np.median(xdata[:, 2])])
medians_y = np.array([np.median(ydata[:, 0]),
                      np.median(ydata[:, 1])])

# Normalize all at once
xarray = xdata / medians_x
yarray = ydata / medians_y

# Print if needed
print(medians_x[0], medians_x[1], medians_x[2], medians_y[0], medians_y[1])
```

```
print(xarray)
print(yarray)
```

```
[[318.0, 0.0, 500.0], [318.0, 0.0, 1000.0], [318.0, 0.0, 1500.0], [318.0, 0.
0, 2000.0], [318.0, 0.0, 2500.0], [318.0, 0.25, 500.0], [318.0, 0.25, 1000.
0], [318.0, 0.25, 1500.0], [318.0, 0.25, 2000.0], [318.0, 0.25, 2500.0], [31
8.0, 0.5, 500.0], [318.0, 0.5, 1000.0], [318.0, 0.5, 1500.0], [318.0, 0.5, 2
000.0], [318.0, 0.5, 2501.0], [303.0, 0.0, 500.0], [303.0, 0.0, 1000.0], [30
3.0, 0.0, 1500.0], [303.0, 0.0, 2000.0], [303.0, 0.0, 2500.0], [303.0, 0.25,
500.0], [303.0, 0.25, 1000.0], [303.0, 0.25, 1500.0], [303.0, 0.25, 2000.0],
[303.0, 0.25, 2500.0], [303.0, 0.5, 500.0], [303.0, 0.5, 1000.0], [303.0, 0.
5, 1500.0], [303.0, 0.5, 2000.0], [303.0, 0.5, 2500.0], [288.0, 0.0, 500.0],
[288.0, 0.0, 1000.0], [288.0, 0.0, 1500.0], [288.0, 0.0, 2000.0], [288.0, 0.
0, 2500.0], [288.0, 0.25, 500.0], [288.0, 0.25, 1000.0], [288.0, 0.25, 1500.
0], [288.0, 0.25, 2000.0], [288.0, 0.25, 2500.0], [288.0, 0.51, 500.0], [28
8.0, 0.5, 1000.0], [288.0, 0.5, 1500.0], [288.0, 0.5, 2000.0], [288.0, 0.5,
2500.0], [268.0, 0.0, 500.0], [268.0, 0.0, 1000.0], [268.0, 0.0, 1500.0], [2
68.0, 0.0, 2000.0], [268.0, 0.0, 2500.0], [268.0, 0.25, 500.0], [268.0, 0.2
5, 1000.0], [268.0, 0.25, 1500.0], [268.0, 0.25, 2000.0], [268.0, 0.25, 250
0.0], [268.0, 0.5, 500.0], [268.0, 0.5, 1000.0], [268.0, 0.5, 1500.0], [268.
0, 0.5, 2000.0], [268.0, 0.5, 2500.0]]
[[35.1316, 0.3808], [40.3764, 0.38686], [47.462, 0.393], [57.5639, 0.39949],
[73.1286, 0.40612], [49.111, 0.4023], [56.4428, 0.40605], [66.3479, 0.4098],
[80.4695, 0.4133], [102.2276, 0.4175], [63.0904, 0.4154], [72.5092, 0.4175],
[85.2338, 0.4197], [103.375, 0.42192], [131.3266, 0.4242], [34.273, 0.3952],
[38.99026, 0.4012], [45.2133, 0.4073], [53.8, 0.4136], [66.413, 0.4201], [4
7.922, 0.4178], [54.518, 0.4215], [63.22, 0.4252], [75.226, 0.429], [92.862,
0.4329], [61.572, 0.4315], [70.0468, 0.43373], [81.226, 0.43597], [96.653,
0.4382], [119.3124, 0.44045], [33.4521, 0.40913], [37.6911, 0.415], [43.160
2, 0.4209], [50.4858, 0.4271], [60.8067, 0.4334], [46.7865, 0.4328], [52.715
1, 0.43646], [60.36425, 0.44016], [70.6099, 0.443926], [85.0447, 0.4477], [6
0.1208, 0.44721], [67.7391, 0.4494], [77.5683, 0.4516], [90.7341, 0.4538],
[109.2828, 0.456], [32.4123, 0.42694], [36.0807, 0.4325], [40.6854, 0.4383],
[46.6374, 0.4442], [54.6293, 0.4503], [45.3472, 0.4519], [50.4796, 0.4555],
[56.9219, 0.4591], [65.2492, 0.4628], [76.4304, 0.4665], [58.2822, 0.4672],
[64.8785, 0.4693], [73.1584, 0.4715], [83.861, 0.4738], [98.2316, 0.476]]
295.5 0.25 1500.0 61.189350000000005 0.432
[[1.07614213 0.         0.33333333]
 [1.07614213 0.         0.66666667]
 [1.07614213 0.         1.        ]
 [1.07614213 0.         1.33333333]
 [1.07614213 0.         1.66666667]
 [1.07614213 1.         0.33333333]
 [1.07614213 1.         0.66666667]
 [1.07614213 1.         1.        ]
 [1.07614213 1.         1.33333333]
 [1.07614213 1.         1.66666667]
 [1.07614213 2.         0.33333333]
 [1.07614213 2.         0.66666667]
 [1.07614213 2.         1.        ]
 [1.07614213 2.         1.33333333]
 [1.07614213 2.         1.66733333]
 [1.02538071 0.         0.33333333]
 [1.02538071 0.         0.66666667]
 [1.02538071 0.         1.        ]
 [1.02538071 0.         1.33333333]
 [1.02538071 0.         1.66666667]
 [1.02538071 1.         0.33333333]
 [1.02538071 1.         0.66666667]
```

```
 [1.02538071 1.         1.        ]
 [1.02538071 1.         1.33333333]
 [1.02538071 1.         1.66666667]
 [1.02538071 2.         0.33333333]
 [1.02538071 2.         0.66666667]
 [1.02538071 2.         1.        ]
 [1.02538071 2.         1.33333333]
 [1.02538071 2.         1.66666667]
 [0.97461929 0.         0.33333333]
 [0.97461929 0.         0.66666667]
 [0.97461929 0.         1.        ]
 [0.97461929 0.         1.33333333]
 [0.97461929 0.         1.66666667]
 [0.97461929 1.         0.33333333]
 [0.97461929 1.         0.66666667]
 [0.97461929 1.         1.        ]
 [0.97461929 1.         1.33333333]
 [0.97461929 1.         1.66666667]
 [0.97461929 2.04       0.33333333]
 [0.97461929 2.         0.66666667]
 [0.97461929 2.         1.        ]
 [0.97461929 2.         1.33333333]
 [0.97461929 2.         1.66666667]
 [0.90693739 0.         0.33333333]
 [0.90693739 0.         0.66666667]
 [0.90693739 0.         1.        ]
 [0.90693739 0.         1.33333333]
 [0.90693739 0.         1.66666667]
 [0.90693739 1.         0.33333333]
 [0.90693739 1.         0.66666667]
 [0.90693739 1.         1.        ]
 [0.90693739 1.         1.33333333]
 [0.90693739 1.         1.66666667]
 [0.90693739 2.         0.33333333]
 [0.90693739 2.         0.66666667]
 [0.90693739 2.         1.        ]
 [0.90693739 2.         1.33333333]
 [0.90693739 2.         1.66666667]]
[[0.57414566 0.88148148]
 [0.65985993 0.89550926]
 [0.77565786 0.90972222]
 [0.94075031 0.92474537]
 [1.19511974 0.94009259]
 [0.80260699 0.93125   ]
 [0.92242849 0.93993056]
 [1.0843047  0.94861111]
 [1.31508996 0.95671296]
 [1.67067635 0.96643519]
 [1.03106831 0.96157407]
 [1.18499706 0.96643519]
 [1.39295155 0.97152778]
 [1.68942798 0.97666667]
 [2.14623296 0.98194444]
 [0.56011381 0.91481481]
 [0.63720664 0.9287037 ]
 [0.738908   0.94282407]
```

```
[0.87923797 0.95740741]
[1.08536861 0.9724537 ]
[0.7831755  0.96712963]
[0.89097204 0.97569444]
[1.03318633 0.98425926]
[1.22939695 0.99305556]
[1.51761704 1.00208333]
[1.00625354 0.99884259]
[1.14475477 1.00400463]
[1.32745323 1.00918981]
[1.57957226 1.01435185]
[1.94988834 1.01956019]
[0.54669808 0.94706019]
[0.61597484 0.96064815]
[0.70535477 0.97430556]
[0.82507495 0.98865741]
[0.99374646 1.00324074]
[0.76461835 1.00185185]
[0.86150776 1.01032407]
[0.98651563 1.01888889]
[1.15395735 1.02760648]
[1.38986114 1.03634259]
[0.98253699 1.03520833]
[1.10704069 1.04027778]
[1.26767648 1.04537037]
[1.48284138 1.05046296]
[1.78597746 1.05555556]
[0.52970492 0.98828704]
[0.58965653 1.00115741]
[0.66490982 1.01458333]
[0.76218165 1.02824074]
[0.89279098 1.04236111]
[0.74109629 1.04606481]
[0.82497363 1.05439815]
[0.93025829 1.06273148]
[1.06634896 1.0712963 ]
[1.24908011 1.07986111]
[0.95248928 1.08148148]
[1.06029072 1.08634259]
[1.19560675 1.09143519]
[1.37051627 1.09675926]
[1.60537087 1.10185185]]
```

In [9]:

```
#===define fitness function used in Genetic Algorithm - added ga_instance ir
#      In PyGAD 2.20.0, the fitness function must accept 3 parameters:
#      1) The instance of the 'pygad.GA' class.
#      2) A solution to calculate its fitness value.
#      3) The solution's index within the population.

def fitness_func(ga_instance, solution, sol_idx):
    global xarray, yarray, keras_ga, model

    model_weights_matrix = pygad.kerasga.model_weights_as_matrix(model=model
                                                                 weights_vec

    model.set_weights(weights=model_weights_matrix)
```

```
    predictions = model.predict(xarray)

    mae = tensorflow.keras.losses.MeanAbsoluteError()
    #data output array and predictions array are provided
    #-->error for each data point is calcuated and mean abs error is compute
    #thus a mean fitness is determined for each solution using all data poin
    meanabs_error = mae(yarray, predictions).numpy() + 0.00000001
    solution_fitness = 1.0/meanabs_error

    return solution_fitness

#========================END

#===define callback that keeps track of best solution for each generation an
#      of all of them over the generations analyzed
def callback_generation(ga_instance):
    print("Generation = {generation}".format(generation=ga_instance.generati
    print("Fitness    = {fitness}".format(fitness=ga_instance.best_solution(

#========================END


#===defining a sequential Neural Network

#initialize weights with values between minval and maxval
initializer = tensorflow.keras.initializers.RandomUniform(minval= -0.9, maxv

model = tensorflow.keras.Sequential([
    tensorflow.keras.layers.Dense(4, activation='relu', input_shape=[3], ker
    tensorflow.keras.layers.Dense(8, activation='relu', kernel_initializer=i
    tensorflow.keras.layers.Dense(4, activation='relu', kernel_initializer=i
    tensorflow.keras.layers.Dense(2, kernel_initializer=initializer)
  ])
#Print summary of model features
model.summary()


#===define a vector that contains all the weights for Neural Network model
weights_vector = pygad.kerasga.model_weights_as_vector(model=model)


#===KerasGA Info below
'''KerasGA is part of the PyGAD library for training Keras models using the
The KerasGA project has a single module named kerasga.py which has a class r
preparing an initial population of Keras model parameters.

PyGAD is an open-source Python library for building the genetic algorithm
and training machine learning algorithms.
Check the library's documentation at Read The Docs: https://pygad.readthedoc

#===Appears to instantiate a keras genetic algorithm object in which the gen
#      NN model, and the population is 40 solutions - just used to set initi
keras_ga = pygad.kerasga.KerasGA(model=model,
                                 num_solutions=40)
```

```
#===set number of generations to run, and number of best fitness solutions t
#      keep and mate in each generation
num_generations = 10
num_parents_mating = 7      #Number of solutions to be selected as parents.

'''sol_per_pop = number of best solutions kept??'''
sol_per_pop = 37  # if fitness_batch_size is supported to calculate the fitn
                  # then the solutions are grouped into batches of size

parent_selection_type = "sss"
'''Steady State Selection
In every generation few chromosomes are selected
(good - with high fitness) for creating a new offspring.
Then some (bad - with low fitness) chromosomes are removed
and the new offspring is placed in their place.
The rest of population survives to new generation.'''

keep_parents = 5
''' keep_parents=-1: Number of parents to keep in the current population.
    -1 (default) means to keep all parents in the next population.
    0 means keep no parents in the next population. A value greater than 0
    means keeps the specified number of parents in the next population. '''

crossover_type = "single_point"
''' crossover_type="single_point": Type of the crossover operation. Supporte
    (for single-point crossover), two_points (for two points crossover), uni
    and scattered (for scattered crossover). Scattered crossover is supporte
    It defaults to single_point.'''

mutation_type = "random"
''' mutation_type="random": Type of the mutation operation. Supported types
    swap (for swap mutation), inversion (for inversion mutation), scramble (
    adaptive (for adaptive mutation). It defaults to random.'''

mutation_percent_genes = "default"
''' mutation_percent_genes="default": Percentage of genes to mutate. It defa
    which is later translated into the integer 10 which means 10% of the ger
    It must be >0 and <=100. Out of this percentage, the number of genes to
    which is assigned to the mutation_num_genes parameter.'''


#===set initial population as the neural network weights
initial_population = keras_ga.population_weights


#===instantiate a pygad genetic algorithm object with desired attributes
ga_instance = pygad.GA(num_generations=num_generations,
                       num_parents_mating=num_parents_mating,
                       initial_population=initial_population,
                       fitness_func=fitness_func,
                       sol_per_pop=sol_per_pop,
                       parent_selection_type = parent_selection_type,
                       keep_parents = keep_parents,
```

```
                        crossover_type = crossover_type,
                        mutation_type = mutation_type,
                        mutation_percent_genes = mutation_percent_genes,
                        on_generation=callback_generation)


#===run this instance of GA object that trains NN
ga_instance.run()

#===the following just plots and prints results after run for specified numb
# After the generations complete, some plots are showed that summarize how t
ga_instance.plot_fitness(title="PyGAD & Keras - Iteration vs. Fitness", line

# Returning the details of the best solution.
solution, solution_fitness, solution_idx = ga_instance.best_solution()
print("Fitness value of the best solution = {solution_fitness}".format(solut
print("Index of the best solution : {solution_idx}".format(solution_idx=solu

# Fetch the parameters of the best solution.
best_solution_weights = pygad.kerasga.model_weights_as_matrix(model=model,
                                                              weights_vector
model.set_weights(best_solution_weights)
predictions = model.predict(xarray)
print("Predictions : \n", predictions)

print("data_outputs : \n", yarray)

mae = tensorflow.keras.losses.MeanAbsoluteError()
abs_error = mae(yarray, predictions).numpy()
print("Absolute Error : ", abs_error)


#saving results in csv

# Your existing code...
solution, solution_fitness, solution_idx = ga_instance.best_solution()
# ... rest of your code ...

# Create a new row
new_row = {
    'Fitness Value': solution_fitness,
    'Absolute Error': abs_error,
    'Solution Index': solution_idx
}

# Append to CSV (create if doesn't exist)
try:
    df = pd.read_csv('results.csv')
    df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)
except FileNotFoundError:
    df = pd.DataFrame([new_row])

df.to_csv('results.csv', index=False)
#=======================END
```

```
/global/home/users/melius/.conda/envs/my_kernel/lib/python3.11/site-packages/keras/src/layers/core/dense.py:92: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_2"

Layer (type)	Output Shape	Par
dense_8 (Dense)	(None, 4)	
dense_9 (Dense)	(None, 8)	
dense_10 (Dense)	(None, 4)	
dense_11 (Dense)	(None, 2)	

Total params: 102 (408.00 B)

Trainable params: 102 (408.00 B)

Non-trainable params: 0 (0.00 B)

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
Generation = 1
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
Fitness    = 2.7857990223212217
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
Generation = 2
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
Fitness    = 3.3241432876778148
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
Generation = 3
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
Fitness    = 3.497294756908367
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
Generation = 4
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
Fitness    = 4.684567508949586
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
Generation = 5
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
Fitness    = 5.2818652530399754
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
Generation = 6
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
Fitness    = 6.012328570710082
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
```

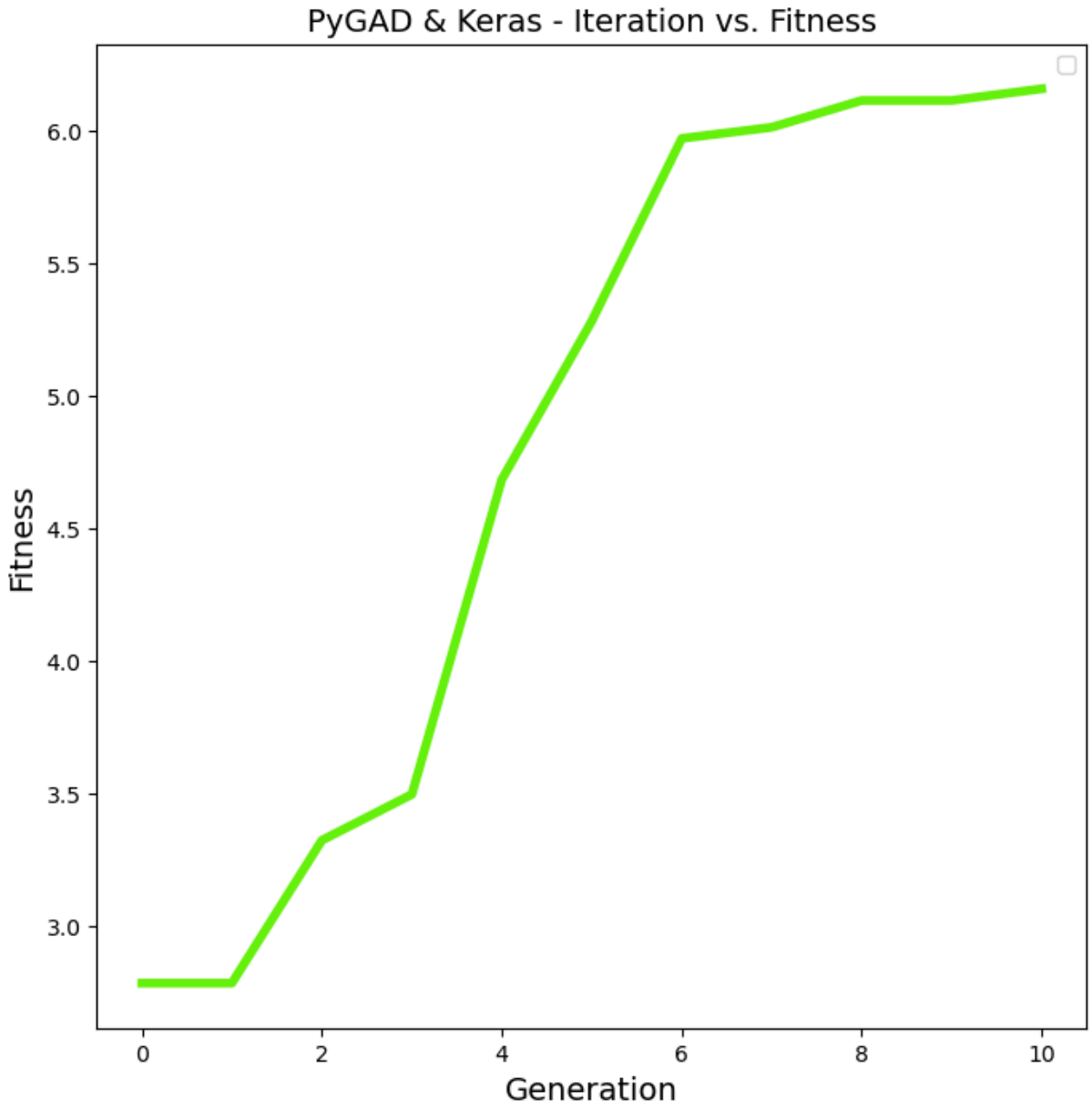
```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
Generation = 8
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
Fitness    = 6.113611794563173
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
Fitness    = 6.113611794563173
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
Generation = 10
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14
```

```
/global/home/users/melius/.conda/envs/my_kernel/lib/python3.11/site-package
s/pygad/visualize/plot.py:120: UserWarning: No artists with labels found to
put in legend.  Note that artists whose label start with an underscore are i
gnored when legend() is called with no argument.
  matplt.legend()
```



```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
Fitness value of the best solution = 6.157518426327663
Index of the best solution : 35
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
Predictions :
 [[0.7699826  1.1146622 ]
 [0.7947252  1.0972776 ]
 [0.8194678  1.0798932 ]
 [0.8441663  1.0625397 ]
 [0.86445254 1.0482862 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [0.7772403  1.1095629 ]
 [0.8019829  1.0921783 ]
 [0.8267255  1.0747938 ]
 [0.85011685 1.0583587 ]
```

```
 [0.8704032  1.0441053 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [0.7844981  1.1044633 ]
 [0.8092407  1.0870789 ]
 [0.8339833  1.0696944 ]
 [0.85606736 1.0541778 ]
 [0.8763536  1.0399244 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [0.79417515 1.0976641 ]
 [0.81891775 1.0802797 ]
 [0.84366035 1.0628952 ]
 [0.8640016  1.048603  ]
 [0.88428783 1.0343497 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]
 [1.0008883  0.9524246 ]]
data_outputs :
 [[0.57414566 0.88148148]
 [0.65985993 0.89550926]
 [0.77565786 0.90972222]
 [0.94075031 0.92474537]
 [1.19511974 0.94009259]
 [0.80260699 0.93125   ]
 [0.92242849 0.93993056]
 [1.0843047  0.94861111]
 [1.31508996 0.95671296]
 [1.67067635 0.96643519]
 [1.03106831 0.96157407]
 [1.18499706 0.96643519]
 [1.39295155 0.97152778]
 [1.68942798 0.97666667]
```

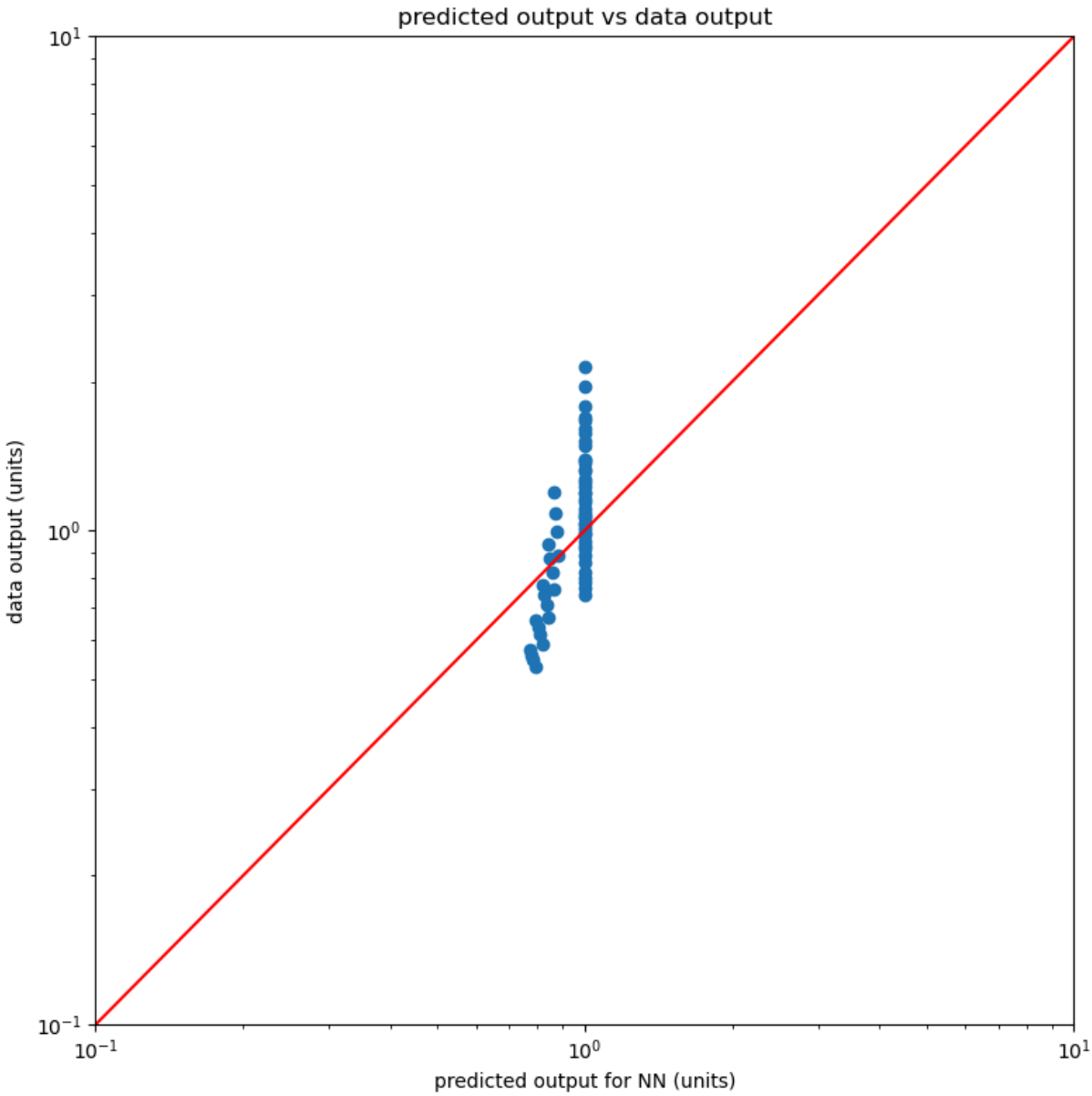
```
 [2.14623296 0.98194444]
 [0.56011381 0.91481481]
 [0.63720664 0.9287037 ]
 [0.738908   0.94282407]
 [0.87923797 0.95740741]
 [1.08536861 0.9724537 ]
 [0.7831755  0.96712963]
 [0.89097204 0.97569444]
 [1.03318633 0.98425926]
 [1.22939695 0.99305556]
 [1.51761704 1.00208333]
 [1.00625354 0.99884259]
 [1.14475477 1.00400463]
 [1.32745323 1.00918981]
 [1.57957226 1.01435185]
 [1.94988834 1.01956019]
 [0.54669808 0.94706019]
 [0.61597484 0.96064815]
 [0.70535477 0.97430556]
 [0.82507495 0.98865741]
 [0.99374646 1.00324074]
 [0.76461835 1.00185185]
 [0.86150776 1.01032407]
 [0.98651563 1.01888889]
 [1.15395735 1.02760648]
 [1.38986114 1.03634259]
 [0.98253699 1.03520833]
 [1.10704069 1.04027778]
 [1.26767648 1.04537037]
 [1.48284138 1.05046296]
 [1.78597746 1.05555556]
 [0.52970492 0.98828704]
 [0.58965653 1.00115741]
 [0.66490982 1.01458333]
 [0.76218165 1.02824074]
 [0.89279098 1.04236111]
 [0.74109629 1.04606481]
 [0.82497363 1.05439815]
 [0.93025829 1.06273148]
 [1.06634896 1.0712963 ]
 [1.24908011 1.07986111]
 [0.95248928 1.08148148]
 [1.06029072 1.08634259]
 [1.19560675 1.09143519]
 [1.37051627 1.09675926]
 [1.60537087 1.10185185]]
Absolute Error :  0.16240308
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
```

In [10]:
```python
#SETTING UP PLOT
#matplotlib inline
# importing the required module
import matplotlib.pyplot as plt
import numpy
plt.rcParams['figure.figsize'] = [8, 8] # for square canvas
#========

'''CALCULATE PREDICTED VALUES, RETRIEVE DATA VALUES AND PLOT'''

plt.scatter(predictions[:, 0], yarray[:, 0])
plt.title('predicted output vs data output')
plt.xlabel('predicted output for NN (units)')
plt.ylabel('data output (units)')
plt.loglog()
plt.xlim(xmax = 10, xmin = 0.1)
plt.ylim(ymax = 10, ymin = 0.1)
# Generate red y=x line
x_data = numpy.linspace(0.1, 10.0, num=3)
y_data = x_data
plt.plot(x_data, y_data, color='red')
plot_fig_title = f"data_output_vs_predicted_output.png"
plt.savefig(plot_fig_title, bbox_inches='tight', dpi=300)
plt.tight_layout()
plt.show()
```


predicted output vs data output
data output (units)
predicted output for NN (units)
10^{1}
10^{0}
10^{-1}
10^{-1}
10^{0}
10^{1}

In [ ]:

In [ ]: